

University Degree in Data Science and Engineering
2022-2026

Bachelor Thesis

“Parallel Programming of FEM in Julia”

Sergio Madrid Cuevas

Sergio Llorente Romano
Luis Emilio García Castillo
Madrid, September 7, 2026



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

ABSTRACT

The field of scientific computing suffers from the dichotomy of iterating quickly on ideas using interactive and dynamically-typed languages, but then implementing final versions of algorithms on rigid and computationally-efficient languages. This division, typically referred to as the two-language problem, forces superfluous workloads on investigators and creates a steep barrier to entrance for students. As a solution to this problem, the Julia programming language was born, offering interactivity without sacrificing speed.

UC3M's GREMA has been suffering from the two-language problem with their in-house electromagnetics FEM solver, HOFEM. For this reason, recent work has focused on introducing Julia into the pipeline, specifically an interface to interact with HOFEM from Julia. This thesis aims to move forward in this regard by implementing a FEM framework in Julia to remove even more friction from this interaction.

For this purpose, a complete framework spanning the entire FEM pipeline was implemented, including a mesh processor to allow arbitrary meshes, rigorous computation and evaluation of basis functions via adaptive numerical integration, and strict system assembly enforcing Dirichlet boundary conditions. To keep the pipeline efficient, parallelization was applied making use of MPI. Verification of the code was performed by ensuring the generated system presented the desired mathematical properties, and by testing whether the magnitude and convergence properties of the approximation error matched the literature. Additionally, finer-grained interaction with HOFEM was explored using the aforementioned interface.

Keywords: Finite Element Method, Computational Electromagnetics, Julia Programming Language, Scientific Computing, High-Performance Computing

CONTENTS

1. INTRODUCTION.	1
1.1. Motivation	1
1.2. Objectives.	2
1.3. Thesis Structure	2
2. STATE OF THE ART.	3
2.1. The Finite Element Method	3
2.1.1. Problem Formulation	3
2.1.2. The Reference Element	7
2.1.3. Finite Element Spaces	10
2.1.4. Problem Discretization	12
2.2. HOFEM.	14
2.3. Julia	15
3. IMPLEMENTATION.	18
3.1. Elements	18
3.2. Meshes	20
3.2.1. Mesh Generation	20
3.2.2. Mesh Loading and Processing	21
3.3. Numerical Integration	28
3.3.1. 1-dimensional Integration	28
3.3.2. 2-dimensional Integration	30
3.3.3. 3-dimensional Integration	31
3.3.4. p -adaptivity.	32
3.4. Computation of the Basis Functions	33
3.5. Assembly of the Global System.	41
3.6. Imposition of the Dirichlet Boundary Conditions	44
3.7. Parallelization with MPI.	47
3.8. Solving the Linear System.	50

4. VERIFICATION	51
4.1. Matrix Properties Test	53
4.1.1. Mass Matrix	53
4.1.2. Stiffness Matrix.	54
4.2. Method of Manufactured Solutions.	56
4.2.1. Functions in the Function Space (Error Test)	60
4.2.2. Functions not in the Function Space (Convergence Test).	61
4.3. HOFEM Test	64
5. REGULATORY FRAMEWORK	67
5.1. Applicable Legislation.	67
5.2. Technical Standards	67
5.3. Intellectual Property	68
6. SOCIOECONOMIC ENVIRONMENT.	69
6.1. Cost of Development.	69
6.2. Socioeconomic Impact.	70
7. CONCLUSIONS	71
7.1. Assessment of Objectives	71
7.2. Limitations	71
7.3. Future Work	72
BIBLIOGRAPHY.	73

LIST OF FIGURES

2.1	Domain boundary partition and tangential trace	5
2.2	Tangential continuity and normal discontinuity of a vector field $\mathbf{E} \in \mathbf{H}(\text{curl}, \mathbb{R}^2)$ across an interface Γ	6
2.3	The reference tetrahedron $\hat{K}$	7
2.4	The barycentric coordinate $\lambda_4(\hat{\mathbf{x}})$ in the reference tetrahedron $\hat{K}$	8
2.5	Tangential component preservation under the covariant Piola transform . .	9
2.6	Degrees of freedom for the second-order Nédélec tetrahedron	12
3.1	Degrees of freedom for two tetrahedra sharing a face	26
3.2	Basis functions for $\hat{K}$ computed using <code>compute_nedelec_coeffs</code>	40
4.1	Test meshes	52
4.2	Fringe plots of the result of running the MMS test 3 on mesh 3 with a polynomial solution	60
4.3	Convergence of the relative error in the MMS test 3	62
4.4	Fringe plots of the result of running the MMS test 3 on mesh 1 with a complex exponential solution	62
4.5	Fringe plots of the result of running the MMS test 3 on mesh 2 with a complex exponential solution	63
4.6	Fringe plots of the result of running the MMS test 3 on mesh 3 with a complex exponential solution	63
4.7	Fringe plots of the result of running the MMS test 3 on mesh 4 with a complex exponential solution	63

LIST OF TABLES

3.1	The .msh file format	22
4.1	Relative errors of the MMS with a polynomial solution	60
4.2	Relative errors of the MMS with an exponential solution	62
6.1	Broken down costs of developing the thesis	70

1. INTRODUCTION

1.1. Motivation

The development of scientific simulations often demands both rapid exploration and high computational throughput, two requirements rarely achievable with a single programming language. Researchers rely on interactive, dynamically-typed languages, such as Python or MATLAB, for quickly prototyping and testing mathematical hypotheses. These tools, however, lack the ability to reach the computational speeds required for deployment-ready numerical simulations, forcing researchers to reimplement the final simulation model in performance-oriented languages, like Fortran or C, which trade ease of development for execution speed. This division became known as the two-language problem.

The two-language problem introduces several issues into the scientific pipeline. First, it forces researchers to maintain two codebases, doubling their workload. Second, it steeply raises the barrier to entry: students and younger investigators must deal with learning a challenging topic and two completely different programming paradigms at the same time. Overall, this makes scientific computing less accessible and slows down progress in the field.

In 2012, a new programming language appeared with the goal of solving this problem, promising the performance of Fortran or C without sacrificing interactivity. This language was Julia [1]. Since then, its popularity has greatly increased within the scientific community, now featuring over 12 000 community-developed packages [2]. Among many others, a promising potential application of Julia is electromagnetics.

HOFEM [3] (higher order finite element method) is a finite element method electromagnetics solver developed by UC3M's GREMA (Group of Radiofrequency, Electromagnetism, Microwaves and Antennas). HOFEM is implemented in Fortran, thus inheriting many of the problems discussed above.

To address the two-language problem, the GREMA has become increasingly interested in Julia. Recent work has focused on generating a Julia-based interface to call HOFEM interactively [4]. This thesis was born, in part, as a continuation of this research line.

Crucially, the goal is to develop a native Julia framework to interact with HOFEM more effectively. To this end, key finite element assembly steps will be recreated in Julia, leveraging its math-like syntax to rigorously compute integrals and basis functions. This will establish a workflow where researchers can prototype using the Julia frontend, and seamlessly call the Fortran backend when needed.

1.2. Objectives

The purpose of this thesis is to develop a mathematically-sound finite element method assembler for electromagnetics in Julia. Specifically, the objectives are:

- **Mesh processing:** Load meshes and generate their local-to-global degrees of freedom (DoF) mapping matrices.
- **Computation of basis functions:** Rigorously compute the basis functions and DoF functionals using numerical integration.
- **Global system assembly:** Programmatically assemble the global linear system in an orientation-consistent manner and appropriately enforce Dirichlet boundary conditions.
- **Parallelization:** Implement process parallelism using MPI to ensure the solver is high-performance computing (HPC) ready.
- **Verification:** Test the mathematical properties of the implementation against proven theoretical results to ensure its correctness.

In addition to the solver, a secondary objective is to test the interaction with HOFEM using the recently implemented Julia interface [4]. To this end, step-by-step execution of HOFEM through the interface will be tested, and a Julia-assembled system will be solved using HOFEM.

1.3. Thesis Structure

This thesis is organized in six sections, aside from the introduction.

Section 2 introduces the theoretical foundation of the finite element method, applied to electromagnetics. It also introduces HOFEM and Julia, and gives a short rundown of the features of the latter.

Section 3 follows the computational pipeline of the developed implementation, including code snippets and explanations of the relevant algorithms.

Section 4 describes the tests performed on the code, as well as their expected and obtained results.

Sections 5 and 6 delve, respectively, into the regulatory framework and the socio-economic environment in which the thesis was developed.

Finally, section 7 shortly assesses the completion of all objectives, and provides a discussion on potential improvements and future lines of work.

2. STATE OF THE ART

2.1. The Finite Element Method

The finite element method (FEM) is a numerical technique developed for solving partial differential equations (PDEs). The method discretizes the domain $\Omega \subset \mathbb{R}^3$ into a finite number of subdomains called elements, and the solution is approximated by a linear combination of basis functions defined on each element. The FEM is widely used in engineering and physics for solving problems where the governing equations are too complex to solve analytically [5]–[7].

2.1.1. Problem Formulation

In the field of electromagnetics, Maxwell's equations are used to describe the behavior of electric and magnetic fields. We assume the system is in a time-harmonic state with angular frequency $\omega \in \mathbb{R}^+$ [8], [9]. This implies that the physical electric, magnetic, electric current, and magnetic current field intensities, $\mathcal{E}, \mathcal{H}, \mathcal{J}, \mathcal{M} : \Omega \times T \rightarrow \mathbb{R}^3$ respectively, can be expressed as complex vector fields $\mathbf{E}, \mathbf{H}, \mathbf{J}, \mathbf{M} : \Omega \rightarrow \mathbb{C}^3$, with a time dependence of the form $e^{i\omega t}$, i.e., $\mathcal{E}(\mathbf{x}, t) = \text{Re}\{\mathbf{E}(\mathbf{x}) e^{i\omega t}\}$, and similarly for $\mathbf{H}, \mathbf{J}$ and $\mathbf{M}$. Here, i is the imaginary unit, e is Euler's number, and $T = \mathbb{R}^+$ is the time domain. Consequently, the time derivative $\partial/\partial t$ becomes a multiplication by $i\omega$ in the frequency domain [9]:

$$\frac{\partial}{\partial t} \mathcal{E}(\mathbf{x}, t) = \text{Re}\{i\omega \mathbf{E}(\mathbf{x}) e^{i\omega t}\}.$$

Then, the time-harmonic Maxwell's equations, with constitutive parameters $\boldsymbol{\epsilon} \in \mathbb{C}^{3 \times 3}$ (permittivity tensor) and $\boldsymbol{\mu} \in \mathbb{C}^{3 \times 3}$ (permeability tensor), are given by:

$$\nabla \times \mathbf{E} = -i\omega \boldsymbol{\mu} \mathbf{H} - \mathbf{M}, \quad (2.1)$$

$$\nabla \times \mathbf{H} = i\omega \boldsymbol{\epsilon} \mathbf{E} + \mathbf{J}, \quad (2.2)$$

$$\nabla \cdot (\boldsymbol{\epsilon} \mathbf{E}) = \rho_e, \quad (2.3)$$

$$\nabla \cdot (\boldsymbol{\mu} \mathbf{H}) = \rho_m, \quad (2.4)$$

where $\rho_e, \rho_m : \Omega \rightarrow \mathbb{C}$ are the electric and magnetic charge densities, respectively. When $\omega \neq 0$, the divergence equations (2.3) and (2.4) are redundant, they are automatically satisfied if the curl equations (2.1) and (2.2) hold and the sources satisfy the continuity equations $\nabla \cdot \mathbf{J} = -i\omega \rho_e$ and $\nabla \cdot \mathbf{M} = -i\omega \rho_m$ [8], [9]. Therefore, we focus on the curl equations to derive the vector wave equation.

We first solve equation (2.1) for $\mathbf{H}$:

$$\mathbf{H} = -\frac{1}{i\omega} \boldsymbol{\mu}^{-1} (\nabla \times \mathbf{E} + \mathbf{M}).$$

Substituting this expression into equation (2.2) we obtain:

$$\nabla \times \left(-\frac{1}{i\omega} \boldsymbol{\mu}^{-1} (\nabla \times \mathbf{E} + \mathbf{M}) \right) = i\omega \boldsymbol{\varepsilon} \mathbf{E} + \mathbf{J}.$$

Multiplying both sides by $-i\omega$ and rearranging terms yields the double-curl equation [9]–[11]:

$$\nabla \times (\boldsymbol{\mu}^{-1} (\nabla \times \mathbf{E})) - \omega^2 \boldsymbol{\varepsilon} \mathbf{E} = -i\omega \mathbf{J} - \nabla \times (\boldsymbol{\mu}^{-1} \mathbf{M}).$$

It is typical to normalize the material parameters relative to free space [10], [12]. Let $\boldsymbol{\varepsilon} = \varepsilon_0 \boldsymbol{\varepsilon}_r$ and $\boldsymbol{\mu} = \mu_0 \boldsymbol{\mu}_r$, where:

- $\varepsilon_0, \mu_0 \in \mathbb{R}^+$ are the permittivity and permeability of free space, respectively, and
- $\boldsymbol{\varepsilon}_r, \boldsymbol{\mu}_r \in \mathbb{C}^{3 \times 3}$ are the relative permittivity and permeability tensors, respectively.

Using the free-space wave-number $k_0 = \omega \sqrt{\varepsilon_0 \mu_0}$ and the free-space impedance $\eta_0 = \sqrt{\mu_0 / \varepsilon_0}$, the final vector wave equation governing the electric field is:

$$\nabla \times (\boldsymbol{\mu}_r^{-1} (\nabla \times \mathbf{E})) - k_0^2 \boldsymbol{\varepsilon}_r \mathbf{E} = -ik_0 \eta_0 \mathbf{J} - \nabla \times (\boldsymbol{\mu}_r^{-1} \mathbf{M}). \quad (2.5)$$

Equation (2.5) represents the strong form of the boundary value problem for the electric field $\mathbf{E}$. For the magnetic field $\mathbf{H}$, we can derive a similar vector wave equation by solving equation (2.2) for $\mathbf{E}$ and substituting it into equation (2.1). The resulting equation is:

$$\nabla \times (\boldsymbol{\varepsilon}_r^{-1} (\nabla \times \mathbf{H})) - k_0^2 \boldsymbol{\mu}_r \mathbf{H} = \nabla \times (\boldsymbol{\varepsilon}_r^{-1} \mathbf{J}) - \frac{ik_0}{\eta_0} \mathbf{M}. \quad (2.6)$$

In this thesis we will focus on the electric field formulation, but the magnetic field formulation can easily be derived following similar steps.

In order for equation (2.5) to have a unique solution, appropriate boundary conditions must be specified on $\Gamma = \partial\Omega$. We consider the following [9], [10]:

- **Dirichlet boundary conditions:** $\mathbf{n} \times \mathbf{E} = \mathbf{g}_D$ on Γ_D ,
- **Neumann boundary conditions:** $\mathbf{n} \times (\boldsymbol{\mu}_r^{-1} (\nabla \times \mathbf{E})) = \mathbf{g}_N$ on Γ_N ,
- **Cauchy boundary conditions:** $\mathbf{n} \times (\boldsymbol{\mu}_r^{-1} (\nabla \times \mathbf{E})) - ik_0 \eta_0 \mathbf{n} \times (\mathbf{n} \times \mathbf{E}) = \mathbf{g}_C$ on Γ_C ,

where $\mathbf{n}$ is the outward unit normal vector to the boundary, $\mathbf{g}_D : \Gamma_D \rightarrow \mathbb{C}^3$, $\mathbf{g}_N : \Gamma_N \rightarrow \mathbb{C}^3$, and $\mathbf{g}_C : \Gamma_C \rightarrow \mathbb{C}^3$ are given vector fields that must be purely tangential to Γ (i.e., $\mathbf{g}_D \cdot \mathbf{n} = 0$, $\mathbf{g}_N \cdot \mathbf{n} = 0$ and $\mathbf{g}_C \cdot \mathbf{n} = 0$), and $\Gamma = \Gamma_D \cup \Gamma_N \cup \Gamma_C$ and are disjoint, as can be seen in figure 2.1.

The FEM solves the weak (or variational) formulation of the boundary value problem. To derive this form, we multiply equation (2.5) by the complex conjugate of a test function

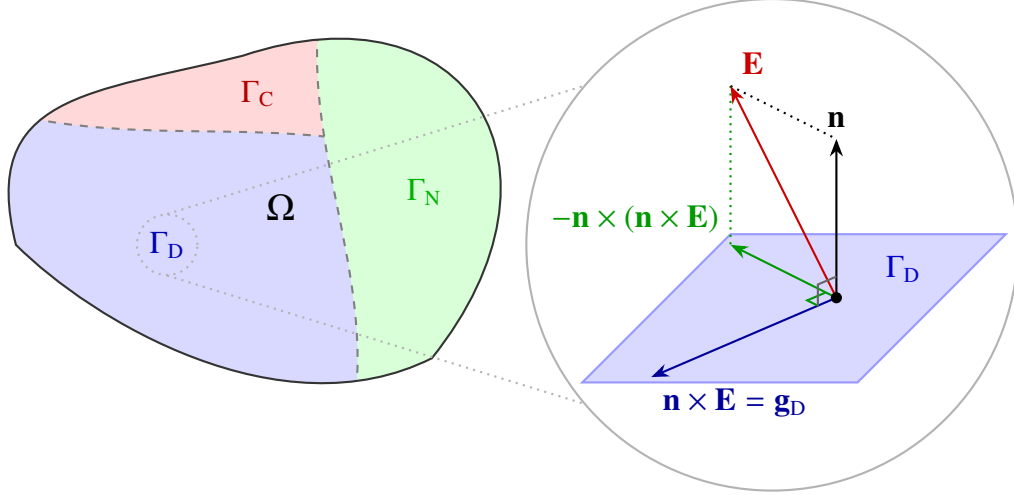


Fig. 2.1. Partition of the domain boundary Γ and local geometric interpretation of the tangential trace $\mathbf{n} \times \mathbf{E}$ on the Dirichlet boundary Γ_D .

$\mathbf{v}$, which belongs to a Hilbert space $\mathbf{V}(\Omega)$ to be defined later. Integrating over the domain Ω yields:

$$\int_{\Omega} \bar{\mathbf{v}} \cdot [\nabla \times (\mu_r^{-1}(\nabla \times \mathbf{E}))] dV - k_0^2 \int_{\Omega} \bar{\mathbf{v}} \cdot (\epsilon_r \mathbf{E}) dV = -ik_0\eta_0 \int_{\Omega} \bar{\mathbf{v}} \cdot \mathbf{J} dV - \int_{\Omega} \bar{\mathbf{v}} \cdot [\nabla \times (\mu_r^{-1} \mathbf{M})] dV.$$

Integrating the first term by parts and applying the boundary conditions, we obtain the weak form of the problem [9], [13]:

$$\begin{aligned} \int_{\Omega} (\nabla \times \bar{\mathbf{v}}) \cdot (\mu_r^{-1}(\nabla \times \mathbf{E})) dV - k_0^2 \int_{\Omega} \bar{\mathbf{v}} \cdot (\epsilon_r \mathbf{E}) dV \\ - ik_0\eta_0 \int_{\Gamma_C} (\mathbf{n} \times \bar{\mathbf{v}}) \cdot (\mathbf{n} \times \mathbf{E}) dS = -ik_0\eta_0 \int_{\Omega} \bar{\mathbf{v}} \cdot \mathbf{J} dV - \int_{\Omega} \bar{\mathbf{v}} \cdot [\nabla \times (\mu_r^{-1} \mathbf{M})] dV \\ - \int_{\Gamma_N} \bar{\mathbf{v}} \cdot \mathbf{g}_N dS - \int_{\Gamma_C} \bar{\mathbf{v}} \cdot \mathbf{g}_C dS, \quad \forall \mathbf{v} \in \mathbf{V}(\Omega). \end{aligned} \quad (2.7)$$

Denoting the left-hand side of equation (2.7) as the bilinear form $a(\mathbf{v}, \mathbf{E})$ and the right-hand side as the linear functional $L(\mathbf{v})$, we can express the weak problem as [13]: find $\mathbf{E} \in \mathbf{V}(\Omega)$ such that

$$a(\mathbf{v}, \mathbf{E}) = L(\mathbf{v}), \quad \forall \mathbf{v} \in \mathbf{V}(\Omega).$$

The choice of the Hilbert space $\mathbf{V}(\Omega)$ is dictated by the weak formulation derived in equation (2.7). Inspection of $a(\mathbf{v}, \mathbf{E})$ and $L(\mathbf{v})$ reveals that both $\mathbf{v}$ and $\mathbf{E}$, and their curls, must be square-integrable over the domain Ω , i.e., $\mathbf{v}, \mathbf{E} \in \mathbf{L}^2(\Omega)$ and $\nabla \times \mathbf{v}, \nabla \times \mathbf{E} \in \mathbf{L}^2(\Omega)$. While the standard Sobolev space $\mathbf{H}^1(\Omega) = \{\mathbf{v} \in \mathbf{L}^2(\Omega) \mid \nabla \mathbf{v} \in \mathbf{L}^2(\Omega)\}$ satisfies these requirements, it is not the most general space for this problem as it imposes square-integrability of all first-order partial derivatives, which is stronger than the regularity required by the curl-curl formulation. To relax this constraint, we define the space $\mathbf{H}(\text{curl}, \Omega)$ as:

$$\mathbf{H}(\text{curl}, \Omega) = \{\mathbf{v} \in \mathbf{L}^2(\Omega) \mid \nabla \times \mathbf{v} \in \mathbf{L}^2(\Omega)\},$$

which is the ideal space for the weak formulation of Maxwell's equations [9], [14], [15]. This space is called the curl-conforming space, as it allows for discontinuities in the normal component of the fields across element interfaces (which $\mathbf{H}^1(\Omega)$ did not) while ensuring the tangential components are continuous (see figure 2.2) [14]–[16], properties that are essential for the correct representation of electromagnetic fields.

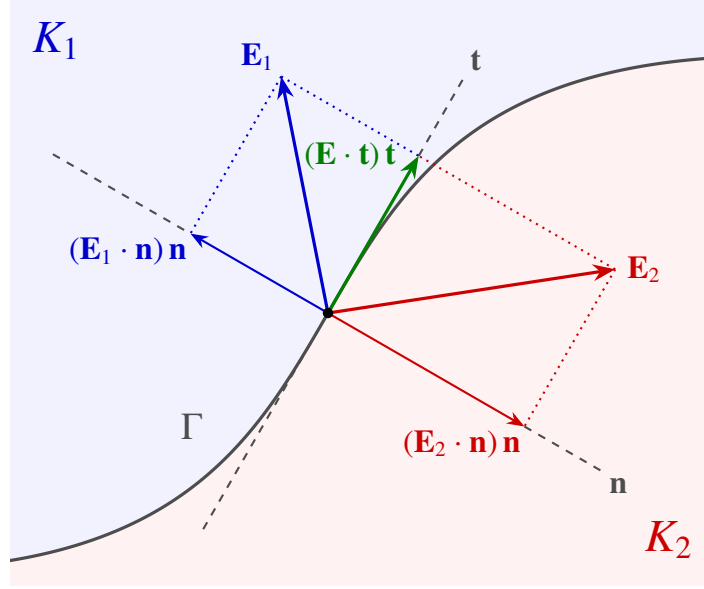


Fig. 2.2. Tangential continuity and normal discontinuity of a vector field $\mathbf{E} \in \mathbf{H}(\text{curl}, \mathbb{R}^2)$ across an interface Γ .

Another important characteristic is that $\mathbf{H}(\text{curl}, \Omega)$ is a Hilbert space when equipped with the inner product $\langle \cdot, \cdot \rangle_{\mathbf{H}(\text{curl}, \Omega)} : \mathbf{H}(\text{curl}, \Omega) \times \mathbf{H}(\text{curl}, \Omega) \rightarrow \mathbb{C}$ defined as [15], [17]:

$$\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{H}(\text{curl}, \Omega)} = \langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{L}^2(\Omega)} + \langle \nabla \times \mathbf{u}, \nabla \times \mathbf{v} \rangle_{\mathbf{L}^2(\Omega)} = \int_{\Omega} \bar{\mathbf{u}} \cdot \mathbf{v} \, dV + \int_{\Omega} (\nabla \times \bar{\mathbf{u}}) \cdot (\nabla \times \mathbf{v}) \, dV,$$

which induces the norm $\|\cdot\|_{\mathbf{H}(\text{curl}, \Omega)} : \mathbf{H}(\text{curl}, \Omega) \rightarrow \mathbb{R}^+$ given by:

$$\|\mathbf{v}\|_{\mathbf{H}(\text{curl}, \Omega)} = \sqrt{\|\mathbf{v}\|_{\mathbf{L}^2(\Omega)}^2 + \|\nabla \times \mathbf{v}\|_{\mathbf{L}^2(\Omega)}^2} = \left(\int_{\Omega} |\mathbf{v}|^2 \, dV + \int_{\Omega} |\nabla \times \mathbf{v}|^2 \, dV \right)^{1/2}.$$

Furthermore, the trace theorems for $\mathbf{H}(\text{curl}, \Omega)$ discussed in [17] ensure that the tangential trace $\mathbf{n} \times \mathbf{v}$ is well-defined on the appropriate boundary trace space, which is essential for the proper imposition of boundary conditions [9].

Let us assume homogeneous Dirichlet boundary conditions (i.e., $\mathbf{g}_D = 0$). Then, we set the trial space for the electric field and the test space as the subspace of functions in $\mathbf{H}(\text{curl}, \Omega)$ whose tangential trace vanishes on the Dirichlet boundary [9], [15]:

$$\mathbf{V}(\Omega) = \{\mathbf{v} \in \mathbf{H}(\text{curl}, \Omega) \mid \mathbf{n} \times \mathbf{v} = 0 \text{ on } \Gamma_D\}.$$

Enforcing $\mathbf{n} \times \mathbf{v} = 0$ allowed us to eliminate the Dirichlet boundary integral from equation (2.7) [9], [13]. Non-homogeneous Dirichlet boundary conditions will be addressed in sections 2.1.4 and 3.6.

2.1.2. The Reference Element

The physical domain Ω can be discretized using various types of elements, such as tetrahedra, hexahedra, prisms, or pyramids [5], [7]. In this thesis, we focus on tetrahedral elements due to their simplicity and flexibility in meshing complex geometries [7], [18]. A tetrahedron is a polyhedron with four triangular faces, four vertices, and six edges. It is uniquely determined by its four vertices $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \mathbf{v}_4 \in \mathbb{R}^3$.

In the FEM framework, a set of basis functions have to be evaluated on each element. To simplify this process, we map each physical element K to a reference element $\hat{K}$ [5], [7], which we define to be the right unit tetrahedron in the local Cartesian coordinate system $\hat{\mathbf{x}} = (\hat{x}, \hat{y}, \hat{z}) \in \mathbb{R}^3$. It is the closed set given by [7]:

$$\hat{K} = \{\hat{\mathbf{x}} \in \mathbb{R}^3 \mid \hat{x}, \hat{y}, \hat{z} \geq 0, \hat{x} + \hat{y} + \hat{z} \leq 1\}.$$

The vertices of $\hat{K}$ are ordered as $\hat{\mathbf{v}}_1 = (0, 0, 0)^\top$, $\hat{\mathbf{v}}_2 = (1, 0, 0)^\top$, $\hat{\mathbf{v}}_3 = (0, 1, 0)^\top$, and $\hat{\mathbf{v}}_4 = (0, 0, 1)^\top$. $\hat{K}$ is shown in figure 2.3.

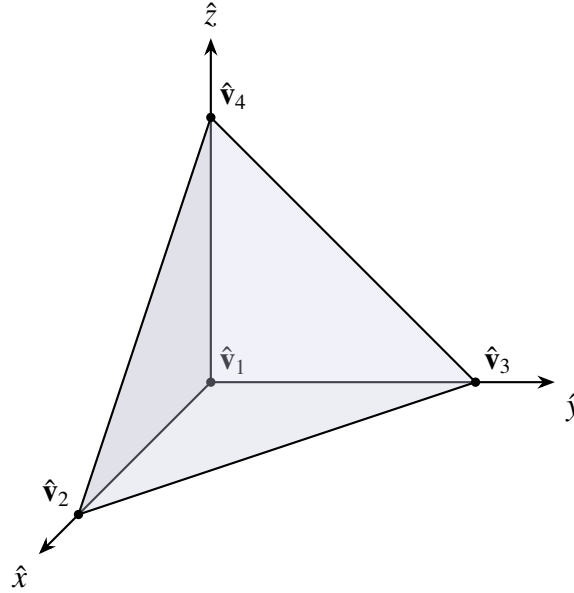


Fig. 2.3. The reference tetrahedron $\hat{K}$.

A fundamental tool for defining transformations on tetrahedra is the system of barycentric coordinates [5], [7], denoted by $\lambda_i : \hat{K} \rightarrow \mathbb{R}$, for $i = 1, \dots, 4$. For any point $\hat{\mathbf{x}} \in \hat{K}$, the value $\lambda_i(\hat{\mathbf{x}})$ represents the weight associated with vertex $\hat{\mathbf{v}}_i$ such that $\hat{\mathbf{x}}$ can be expressed as a convex combination of the vertices:

$$\hat{\mathbf{x}} = \sum_{i=1}^4 \lambda_i(\hat{\mathbf{x}}) \hat{\mathbf{v}}_i,$$

subject to the constraints $\lambda_i(\hat{\mathbf{x}}) \geq 0$ for all i and $\sum_{i=1}^4 \lambda_i(\hat{\mathbf{x}}) = 1$.

For the right unit tetrahedron $\hat{K}$, the barycentric coordinates can be explicitly given in terms of the local Cartesian coordinates as:

$$\lambda_1(\hat{\mathbf{x}}) = 1 - \hat{x} - \hat{y} - \hat{z}, \quad \lambda_2(\hat{\mathbf{x}}) = \hat{x}, \quad \lambda_3(\hat{\mathbf{x}}) = \hat{y}, \quad \lambda_4(\hat{\mathbf{x}}) = \hat{z}.$$

Each of the barycentric coordinates λ_i effectively measures the proportional volume of the sub-tetrahedron formed by the point $\hat{\mathbf{x}}$ and the face opposite to $\hat{\mathbf{v}}_i$, with respect to the volume of the full tetrahedron [7]. For instance, if $\lambda_i(\hat{\mathbf{x}}) = 1$, then $\hat{\mathbf{x}}$ coincides with vertex $\hat{\mathbf{v}}_i$, while if $\lambda_i(\hat{\mathbf{x}}) = 0$, then $\hat{\mathbf{x}}$ lies on the face opposite to vertex $\hat{\mathbf{v}}_i$. This can be visualized in figure 2.4.

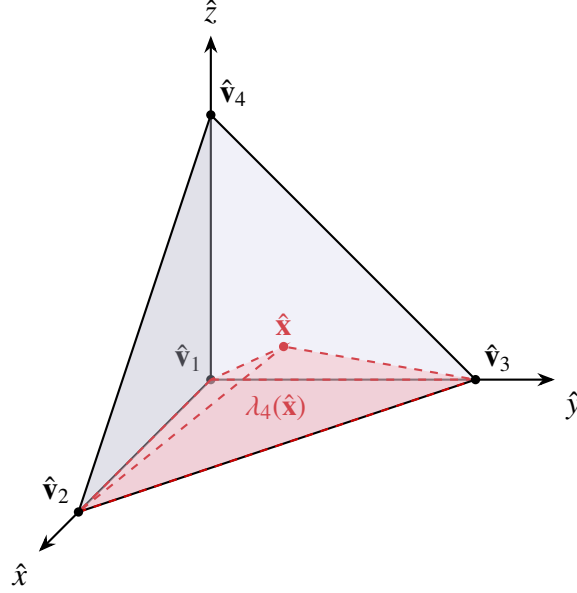


Fig. 2.4. The barycentric coordinate $\lambda_4(\hat{\mathbf{x}})$ in the reference tetrahedron $\hat{K}$.

For every physical element K , we define a bijective mapping $\Phi_K : \hat{K} \rightarrow K$ such that for any $\hat{\mathbf{x}} \in \hat{K}$, the physical coordinate is $\mathbf{x} = \Phi_K(\hat{\mathbf{x}})$ [5], [7]. If the physical element K is a straight-edged tetrahedron defined by vertices $\mathbf{v}_i$, $i = 1, \dots, 4$, the mapping is affine and can be written as [7]:

$$\Phi_K(\hat{\mathbf{x}}) = \sum_{i=1}^4 \lambda_i(\hat{\mathbf{x}}) \mathbf{v}_i = \mathbf{v}_1 + \mathbf{J}_K^T \hat{\mathbf{x}},$$

where $\mathbf{J}_K \in \mathbb{R}^{3 \times 3}$ is the Jacobian matrix of the mapping:

$$\mathbf{J}_K = \frac{\partial \Phi_K}{\partial \hat{\mathbf{x}}} = \begin{bmatrix} \frac{\partial(\Phi_K)_1}{\partial \hat{x}} & \frac{\partial(\Phi_K)_2}{\partial \hat{x}} & \frac{\partial(\Phi_K)_3}{\partial \hat{x}} \\ \frac{\partial(\Phi_K)_1}{\partial \hat{y}} & \frac{\partial(\Phi_K)_2}{\partial \hat{y}} & \frac{\partial(\Phi_K)_3}{\partial \hat{y}} \\ \frac{\partial(\Phi_K)_1}{\partial \hat{z}} & \frac{\partial(\Phi_K)_2}{\partial \hat{z}} & \frac{\partial(\Phi_K)_3}{\partial \hat{z}} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_2 - \mathbf{v}_1 & \mathbf{v}_3 - \mathbf{v}_1 & \mathbf{v}_4 - \mathbf{v}_1 \end{bmatrix}^T.$$

In this thesis we will not deal with curved elements, but it is worth mentioning that for them, the mapping Φ_K is generally nonlinear, and the Jacobian matrix $\mathbf{J}_K$ becomes a function of the reference coordinates $\hat{\mathbf{x}}$.

To perform integration over the physical element K , we utilize the change of variables formula, which relates the volume element dV in the physical coordinates to the volume element $d\hat{V}$ in the reference coordinates:

$$\int_K f dV = \int_{\hat{K}} f \circ \Phi_K \det(\mathbf{J}_K) d\hat{V}.$$

For the change of variables to be valid (orientation preserving and non-singular), it is required that $\det(\mathbf{J}_K) > 0$ for all elements K in the mesh. We can assume this condition is satisfied without loss of generality [7].

A critical aspect of the FEM in electromagnetics is that vector fields in $\mathbf{H}(\text{curl}, \Omega)$ cannot be mapped using the standard coordinate change, $\mathbf{v} = \hat{\mathbf{v}} \circ \Phi_K^{-1}$, as it fails to preserve the tangential continuity of the field across element interfaces and destroys the curl-conformity of the space [14]. Instead, we use the covariant Piola transformation [14]–[16]. Let $\hat{\mathbf{E}} : \hat{K} \rightarrow \mathbb{C}^3$ be a vector field defined on the reference element. The restriction of the field $\mathbf{E}$ to the physical element K is defined as:

$$\mathbf{E}|_K(\mathbf{x}) = \mathbf{J}_K^{-1} \hat{\mathbf{E}}(\hat{\mathbf{x}}), \quad (2.8)$$

where $\hat{\mathbf{x}} = \Phi_K^{-1}(\mathbf{x})$. This transformation is derived from the requirement that the tangential component of the field along a curve $\hat{\gamma} \subset \hat{K}$ maps correctly to the corresponding curve $\gamma \subset K$, $\gamma = \Phi_K(\hat{\gamma})$, such that [14], [15]:

$$\int_{\gamma} \mathbf{E}|_K \cdot d\mathbf{l} = \int_{\hat{\gamma}} \hat{\mathbf{E}} \cdot d\hat{\mathbf{l}}. \quad (2.9)$$

For the specific case where the curve is an edge of the reference element, $\hat{e} = (\hat{\mathbf{v}}_i, \hat{\mathbf{v}}_j)$, with tangent vector $\mathbf{t}_{\hat{e}} = \hat{\mathbf{v}}_j - \hat{\mathbf{v}}_i$, the integral equality in equation (2.9) simplifies to:

$$\mathbf{E}|_K \cdot \mathbf{t} = \hat{\mathbf{E}} \cdot \mathbf{t}_{\hat{e}},$$

with $e = \Phi_K(\hat{e})$ and $\mathbf{t} = \mathbf{J}_K^T \mathbf{t}_{\hat{e}}$. Figure 2.5 provides a geometric interpretation of this result, showing how the transformation maps vectors between elements while preserving the tangential line integral along the corresponding edges.

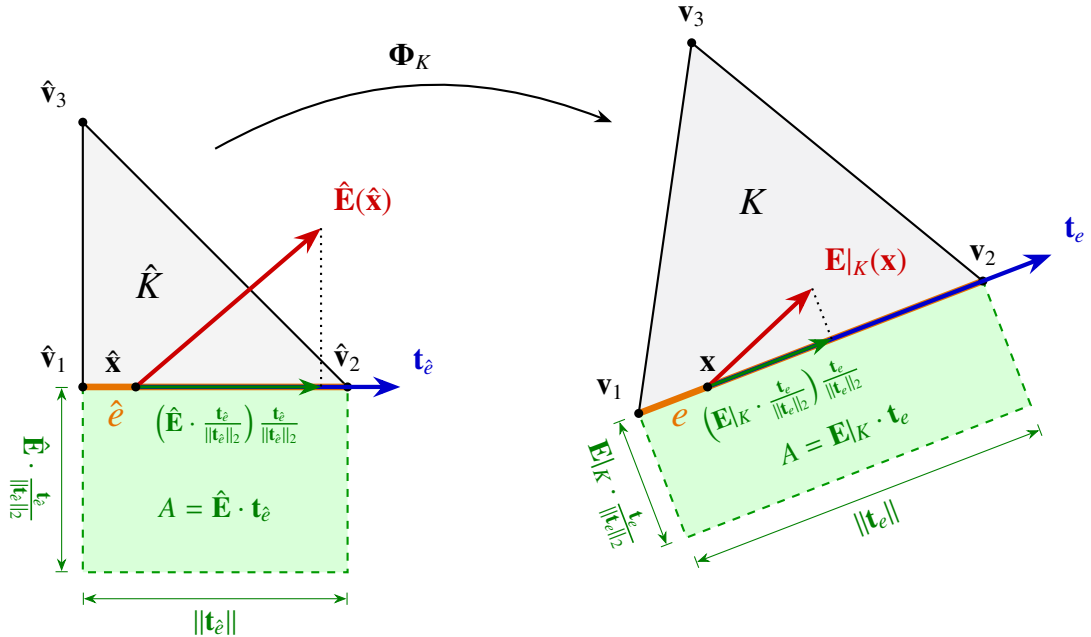


Fig. 2.5. Tangential component preservation under the covariant Piola transform, illustrating the equality $\int_{\hat{e}} \hat{\mathbf{E}} \cdot d\hat{\mathbf{l}} = \int_e \mathbf{E}|_K \cdot d\mathbf{l}$, assuming constant $\hat{\mathbf{E}}$ along $\hat{e}$ for simpler visualization.

The transformation in equation (2.8) satisfies two important properties for Maxwell's equations [9], [16], the first of which was shown in figure 2.5:

- **Preservation of tangential continuity:** if $\hat{\mathbf{E}}$ has a continuous tangential component across a face in the reference element, then $\mathbf{E}|_K$ will preserve this continuity across the corresponding face in the physical element, ensuring $\mathbf{E} \in \mathbf{H}(\text{curl}, \Omega)$.
- **Commutativity with the curl operator:** the curl of the transformed field relates to the reference curl via the contravariant Piola transform:

$$\nabla \times \mathbf{E}|_K(\mathbf{x}) = \frac{1}{\det(\mathbf{J}_K)} \mathbf{J}_K^\top (\hat{\nabla} \times \hat{\mathbf{E}})(\hat{\mathbf{x}}),$$

where $\hat{\nabla} = \left(\frac{\partial}{\partial \hat{x}}, \frac{\partial}{\partial \hat{y}}, \frac{\partial}{\partial \hat{z}} \right)^\top$ is the gradient operator in the reference coordinates [14]–[16].

These properties ensure that the mathematical structure of the differential operators is preserved during the mapping process. This structural compatibility between the spaces is summarized by the exact de Rham complex [19]. In the commutative diagram below, the horizontal arrows represent the differential operators, while the vertical arrows represent the specific coordinate transformations required for each space:

$$\begin{array}{ccccccc} H^1(\hat{K}) & \xrightarrow{\hat{\nabla}} & \mathbf{H}(\text{curl}, \hat{K}) & \xrightarrow{\hat{\nabla} \times} & \mathbf{H}(\text{div}, \hat{K}) & \xrightarrow{\hat{\nabla} \cdot} & L^2(\hat{K}) \\ \circ \Phi_K^{-1} \downarrow & & \downarrow \mathbf{J}_K^{-1} & & \downarrow \frac{1}{\det(\mathbf{J}_K)} \mathbf{J}_K^\top & & \downarrow \frac{1}{\det(\mathbf{J}_K)} \\ H^1(K) & \xrightarrow{\nabla} & \mathbf{H}(\text{curl}, K) & \xrightarrow{\nabla \times} & \mathbf{H}(\text{div}, K) & \xrightarrow{\nabla \cdot} & L^2(K) \end{array} \quad (2.10)$$

where $\mathbf{H}(\text{div}, K) = \{\mathbf{v} \in \mathbf{L}^2(K) \mid \nabla \cdot \mathbf{v} \in L^2(K)\}$ is the divergence-conforming space.

2.1.3. Finite Element Spaces

Following the framework established by Philippe G. Ciarlet [7] and adapted to electromagnetics by Peter Monk [9], [15], we define a finite element as a triple $(K, \mathcal{P}, \Sigma)$, where K is a geometric element, $\mathcal{P}$ is a finite-dimensional function space defined on K , and Σ is a finite set of linear functionals, $\{\ell_i : \mathcal{P} \rightarrow \mathbb{C}\}$, called degrees of freedom (DoFs). The set Σ must be unisolvent with respect to $\mathcal{P}$, i.e., for any set of values $\{\alpha_i\} \subset \mathbb{C}$, there must exist a unique function $p \in \mathcal{P}$ such that $\ell_i(p) = \alpha_i$, for all i , implying $\dim(\mathcal{P}) = |\Sigma|$. This can be understood as the degrees of freedom uniquely determining a function in the function space [7], [14]

To discretize the problem in equation (2.7) using the reference tetrahedron $\hat{K}$, we must define a function space $\mathcal{P}$ that respects $\mathbf{H}(\text{curl}, \Omega)$'s constraints. To construct this space, we first introduce some useful notation [14], [16]. We denote by $\mathbb{P}_k$ the space of polynomials in three variables of total degree at most k :

$$\mathbb{P}_k = \{p \in \mathbb{R}[\hat{x}, \hat{y}, \hat{z}] \mid \deg(p) \leq k\},$$

where $\mathbb{R}[\hat{x}, \hat{y}, \hat{z}]$ denotes the polynomial ring over the variables $\hat{x}$, $\hat{y}$, and $\hat{z}$ with coefficients in $\mathbb{R}$, by $\tilde{\mathbb{P}}_k$ the space of homogeneous polynomials of degree k :

$$\tilde{\mathbb{P}}_k = \{p \in \mathbb{R}[\hat{x}, \hat{y}, \hat{z}] \mid p(\alpha\hat{x}, \alpha\hat{y}, \alpha\hat{z}) = \alpha^k p(\hat{x}, \hat{y}, \hat{z}), \forall \alpha \in \mathbb{R}\},$$

and by $\mathbf{S}_k$ the space of homogeneous vector polynomials of order k orthogonal to the position vector $\hat{\mathbf{x}}$:

$$\mathbf{S}_k = \{\mathbf{p} \in (\tilde{\mathbb{P}}_k)^3 \mid \mathbf{p}(\hat{\mathbf{x}}) \cdot \hat{\mathbf{x}} = 0\}.$$

Then, we define the order p Nédélec space of the first kind, denoted $\mathcal{N}_p$, as [14]:

$$\mathcal{N}_p = (\mathbb{P}_{p-1})^3 \oplus \mathbf{S}_p.$$

Another characterization of the space is [14], [16]:

$$\mathcal{N}_p = \{\mathbf{p} \in (\mathbb{P}_p)^3 \mid \mathbf{p}(\hat{\mathbf{x}}) = \mathbf{a}(\hat{\mathbf{x}}) + \mathbf{b}(\hat{\mathbf{x}}) \times \hat{\mathbf{x}}, \mathbf{a} \in (\mathbb{P}_{p-1})^3, \mathbf{b} \in (\tilde{\mathbb{P}}_{p-1})^3\}.$$

To ensure $\mathcal{N}_p$'s conformity in $\mathbf{H}(\text{curl}, \Omega)$, the degrees of freedom Σ must enforce the continuity of the tangential component $\mathbf{n} \times \mathbf{E}$ across shared interfaces, and the unsolvence condition must hold to ensure a unique representation in $\mathcal{N}_p$. We partition the set Σ based on the topological entities of the tetrahedron and end up with the following DoFs [9], [14]:

- Edge DoFs:

$$\ell_{\hat{e},q}(\mathbf{p}) = \int_{\hat{e}} (\mathbf{p} \cdot \mathbf{t}_{\hat{e}}) q \, dl, \quad \forall q \in \mathbb{P}_{p-1}(\hat{e}),$$

where $\hat{e} = (\hat{\mathbf{v}}_i, \hat{\mathbf{v}}_j)$ is an edge of the tetrahedron,

$$\mathbf{t}_{\hat{e}} = \frac{\hat{\mathbf{v}}_j - \hat{\mathbf{v}}_i}{\|\hat{\mathbf{v}}_j - \hat{\mathbf{v}}_i\|_2}$$

is the unit tangent vector along edge $\hat{e}$, and $\mathbb{P}_{p-1}(\hat{e})$ denotes the polynomials in $\mathbb{P}_{p-1}$ restricted to the edge $\hat{e}$. There are p of these DoFs per edge.

- Face DoFs:

$$\ell_{\hat{f},\mathbf{q}}(\mathbf{p}) = \int_{\hat{f}} (\mathbf{p} \times \mathbf{n}_{\hat{f}}) \cdot \mathbf{q} \, dS, \quad \forall \mathbf{q} \in (\mathbb{P}_{p-2})^2(\hat{f}),$$

where $\hat{f} = (\hat{\mathbf{v}}_i, \hat{\mathbf{v}}_j, \hat{\mathbf{v}}_k)$ is a face of the tetrahedron,

$$\mathbf{n}_{\hat{f}} = \frac{(\hat{\mathbf{v}}_j - \hat{\mathbf{v}}_i) \times (\hat{\mathbf{v}}_k - \hat{\mathbf{v}}_i)}{\|(\hat{\mathbf{v}}_j - \hat{\mathbf{v}}_i) \times (\hat{\mathbf{v}}_k - \hat{\mathbf{v}}_i)\|_2}$$

is the unit normal vector of face $\hat{f}$, and $(\mathbb{P}_{p-2})^2(\hat{f})$ denotes the polynomials in $(\mathbb{P}_{p-2})^2$ restricted to the face $\hat{f}$. There are $p(p-1)$ of these DoFs per face.

- Volume DoFs:

$$\ell_{V,\mathbf{q}}(\mathbf{p}) = \int_{\hat{K}} \mathbf{p} \cdot \mathbf{q} \, dV, \quad \forall \mathbf{q} \in (\mathbb{P}_{p-3})^3(\hat{K}),$$

where $(\mathbb{P}_{p-3})^3(\hat{K})$ denotes the polynomials in $(\mathbb{P}_{p-3})^3$ restricted to $\hat{K}$. There are $\frac{1}{2}p(p-1)(p-2)$ of these DoFs.

As illustrated in figure 2.6, the total dimension of Σ matches $\dim(\mathcal{N}_p) = \frac{1}{2}p(p + 2)(p + 3)$ [9], [14]. More importantly, the edge and face DoFs are defined entirely on boundary data, so, when two elements touch, their respective DoFs coincide, ensuring $\mathcal{N}_p$ respects tangential continuity [9]. Note that volume DoFs cannot coincide across separate elements. Finally, the unisolvence of Σ with respect to $\mathcal{N}_p$ follows from the fact that any $\mathbf{p} \in \mathcal{N}_p$ whose edge, face, and volume DoFs all vanish must be the zero vector field [9], [14].

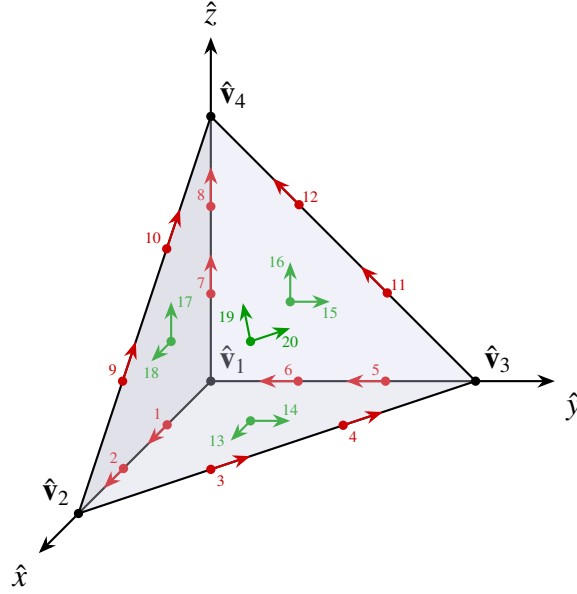


Fig. 2.6. Degrees of freedom for the second-order Nédélec tetrahedron. Edge DoFs (1–12) are represented by red arrows, and face DoFs (13–20) are represented by green arrows.

In this thesis, $(\hat{K}, \mathcal{N}_2, \Sigma)$ will be chosen as the finite element space.

2.1.4. Problem Discretization

The first, and most important, object to discretize in the FEM is the domain Ω . We define a mesh, denoted by $\mathcal{T}_h = \{K_i\}_{i=1}^M$, as a finite set of non-empty geometric elements (in our case tetrahedra) partitioning the closure $\overline{\Omega} = \Omega \cup \Gamma$ [5], [7], [9]. That is:

1. $\mathring{K}_i \cap \mathring{K}_j = \emptyset$ for all $i \neq j$, where $\mathring{K}_i = K_i \setminus \partial K_i$,
2. $\text{diam}(K_i) > 0$ for all i , where $\text{diam}(K_i) = \max_{\mathbf{x}, \mathbf{y} \in K_i} \|\mathbf{x} - \mathbf{y}\|_2$, and
3. $\overline{\Omega} = \bigcup_{i=1}^M K_i$.

For a tetrahedral mesh, statement 1 must be supplemented with the condition that the intersection of two distinct elements, $K_i \cap K_j$, must be either empty or a topological entity

of the tetrahedron, such as a vertex, edge, or face[5], [7]. We denote the mesh size, or refinement level, by h , and define it as:

$$h = \max_{K \in \mathcal{T}_h} \text{diam}(K).$$

For a given mesh $\mathcal{T}_h$, the global discretization of $\mathbf{V}(\Omega)$ is [5], [7], [9]:

$$\mathbf{V}_h(\Omega) = \{\mathbf{v} \in \mathbf{V}(\Omega) \mid \mathbf{v}|_K \in \mathcal{N}_p(K), \forall K \in \mathcal{T}_h\},$$

with $\mathcal{N}_p(K) = \mathbf{J}_K \mathcal{N}_p \circ \Phi_K^{-1}$. Let $\{\mathbf{N}_i\}_{i=1}^N$ be a global basis for $\mathbf{V}_h(\Omega)$. Then, the discretization $\mathbf{E}_h \in \mathbf{V}_h(\Omega)$ of the electric field $\mathbf{E}$ is given by [9], [10]:

$$\mathbf{E}_h = \sum_{i=1}^N g_i \mathbf{N}_i,$$

where the $g_i \in \mathbb{C}$ are the unknowns to be determined. Substituting $\mathbf{E}_h$ into equation (2.7) and choosing the test functions $\mathbf{v}$ from the basis $\{\mathbf{N}_j\}_{j=1}^N$, the weak problem becomes a system of linear equations:

$$\begin{aligned} \sum_{i=1}^N g_i \left(\int_{\Omega} (\nabla \times \mathbf{N}_j) \cdot (\boldsymbol{\mu}_r^{-1} (\nabla \times \mathbf{N}_i)) dV - k_0^2 \int_{\Omega} \mathbf{N}_j \cdot (\boldsymbol{\epsilon}_r \mathbf{N}_i) dV \right) \\ = -ik_0 \eta_0 \int_{\Omega} \mathbf{N}_j \cdot \mathbf{J} dV - \int_{\Omega} \mathbf{N}_j \cdot (\nabla \times (\boldsymbol{\mu}_r^{-1} \mathbf{M})) dV, \end{aligned} \quad (2.11)$$

where the boundary integrals are removed as we will only consider Dirichlet boundary conditions, and complex conjugation is omitted since the basis functions are real-valued, implying $\bar{\mathbf{N}}_i = \mathbf{N}_i$. Separating integrals in equation (2.11) we obtain the matrix equation [9], [10]:

$$(\mathbf{K} - k_0^2 \mathbf{M}) \mathbf{g} = -(ik_0 \eta_0 \mathbf{j} + \mathbf{m}), \quad (2.12)$$

where $\mathbf{K} \in \mathbb{C}^{N \times N}$ is called the stiffness matrix, $\mathbf{M} \in \mathbb{C}^{N \times N}$ is called the mass matrix, and $\mathbf{j}, \mathbf{m} \in \mathbb{C}^N$ are called the source vectors.

Equation (2.12) was derived assuming homogeneous Dirichlet boundary conditions, i.e., $\mathbf{E}_h \in \mathbf{V}_h(\Omega)$. To allow for non-homogeneous Dirichlet boundary conditions, this system is algebraically modified to enforce the true $\mathbf{g}_D$ onto the boundary unknowns (see section 3.6).

In practice, the global matrices defined in equation (2.12) are assembled by summing the contributions from each local element $K \in \mathcal{T}_h$. Each of the integrals is evaluated by pulling them back to the reference tetrahedron $\hat{K}$ using the mappings described in section 2.1.2 [9], [10].

Stiffness Matrix

The stiffness matrix entry corresponds to the inner product of the curled basis functions, weighted by the inverse permeability tensor [9], [15]:

$$(\mathbf{K}_K)_{i,j} = \int_K (\nabla \times \mathbf{N}_j) \cdot (\boldsymbol{\mu}_r^{-1} (\nabla \times \mathbf{N}_i)) dV.$$

We apply the contravariant Piola transform for $\mathbf{H}(\text{div}, \Omega)$ elements, $\nabla \times \mathbf{v} = \frac{1}{\det(\mathbf{J}_K)} \mathbf{J}_K^\top (\hat{\nabla} \times \hat{\mathbf{v}})$, to both the curls of the trial function $\mathbf{N}_i$ and the test function $\mathbf{N}_j$. Substituting these into the integral we get:

$$(\mathbf{K}_K)_{i,j} = \int_{\hat{K}} (\hat{\nabla} \times \hat{\mathbf{N}}_j)^\top \left[\frac{\mathbf{J}_K \boldsymbol{\mu}_r^{-1} \mathbf{J}_K^\top}{\det(\mathbf{J}_K)} \right] (\hat{\nabla} \times \hat{\mathbf{N}}_i) d\hat{V}.$$

Mass Matrix

The mass matrix entry corresponds to the inner product of the basis functions, weighted by the permittivity tensor [9], [10]:

$$(\mathbf{M}_K)_{i,j} = \int_K \mathbf{N}_j \cdot (\boldsymbol{\varepsilon}_r \mathbf{N}_i) dV.$$

We apply the covariant Piola transform for elements of $\mathbf{H}(\text{curl}, \Omega)$, $\mathbf{v} = \mathbf{J}_K^{-1} \hat{\mathbf{v}}$, and obtain:

$$(\mathbf{M}_K)_{i,j} = \int_{\hat{K}} \hat{\mathbf{N}}_j^\top \left[\mathbf{J}_K^{-\top} \det(\mathbf{J}_K) \boldsymbol{\varepsilon}_r \mathbf{J}_K^{-1} \right] \hat{\mathbf{N}}_i d\hat{V}.$$

Source Vectors

The source vector entries are given by [9], [10]:

$$(\mathbf{j}_K)_j = \int_K \mathbf{N}_j \cdot \mathbf{J} dV, \quad (\mathbf{m}_K)_j = \int_K \mathbf{N}_j \cdot \nabla \times (\boldsymbol{\mu}_r^{-1} \mathbf{M}) dV,$$

Applying the covariant Piola transform to the basis functions yields:

$$(\mathbf{j}_K)_j = \int_{\hat{K}} \hat{\mathbf{N}}_j^\top \left[\mathbf{J}_K^{-\top} \det(\mathbf{J}_K) \right] (\mathbf{J} \circ \boldsymbol{\Phi}_K) d\hat{V}, \quad (\mathbf{m}_K)_j = \int_{\hat{K}} \hat{\mathbf{N}}_j^\top \left[\mathbf{J}_K^{-\top} \det(\mathbf{J}_K) \right] (\nabla \times (\boldsymbol{\mu}_r^{-1} \mathbf{M}) \circ \boldsymbol{\Phi}_K) d\hat{V}.$$

2.2. HOFEM

HOFEM [3] is an “in-house parallel electromagnetics simulator based on the FEM”, written in modern Fortran by UC3M’s GREMA. The execution of HOFEM can be carried out in a sequential or parallel regime, making use of MPI (message-passing interface) [20] in the latter. For ease of use, HOFEM also offers a GUI (graphical user interface).

HOFEM can be used to solve closed-domain problems, such as waveguide and microwave circuits, or open-domain problems, like antennas and radar cross sections [3]. At its core, HOFEM uses high-order Nédélec elements of the first kind to solve equations (2.5) and (2.6), for a tetrahedral, prismatic or mixed mesh.

HOFEM suffers from the two-language problem. Being written in Fortran, which is hard to get familiar with, access for less experienced researchers is limited. For this reason, an interface that allows HOFEM to be called interactively from Julia was recently developed [4], but further work is still required to fully make use of Julia’s capabilities.

2.3. Julia

Julia [21] appeared in 2012 [1] as a potential solution to the two-language problem. It is a dynamically-typed and compiled programming language, promising a simple syntax while offering low-level-like performance through its just-in-time (JIT) compiler backed by LLVM [22].

Its main paradigm is based on a concept known as multiple dispatch [2], which is similar to function overloading. Multiple dispatch allows Julia to choose a specific implementation of a function based on the input types at runtime, rather than at compile-time (as is the case for overloading). As an example, let the following snippet define the `sign` function for real and complex types:

```
1 sign(x::Real) = x < 0 ? -one(x) : x > 0 ? one(x) : x
2 sign(z::Complex) = z / abs(z)
```

When calling the function as `sign(-13)`, Julia will automatically choose the implementation with a `Real` parameter.

Another relevant feature of Julia is its hierarchical type system [2]. Julia distinguishes between abstract and concrete types:

- **Abstract types:** They act as conceptual categories and cannot be instantiated. In the example above, `Real` and `Complex` are abstract types. They are generally used for defining groups of types sharing some behavior.
- **Concrete types:** They can be instantiated into values and cannot have any subtypes. `Float64` is an example of a concrete `Real` type.

Type hierarchies can be established using the subtype operator `<:`. Following the example, `Float64 <: Real` would set `Float64` to be a subtype of `Real`. Furthermore, types can be parametrized. That is, we can define a generic type that takes another type as a parameter. This is what allows vectors or matrices to be defined over any numeric type.

Type hierarchies allow function parameter types to be set as abstract types. However, any input must have been previously instantiated and, therefore, must be of a concrete type. This leads into Julia's next useful feature: method specialization [2]. When a parameter is declared with an abstract type (or no type at all, imposing no limits on the possible types that can be passed), Julia creates a specialized implementation for each combination of types used to call the function. For instance, calling `sign` with a `Float64` and later with a `Float32` will result in Julia compiling it twice (once for each type), ensuring compiled functions are always as efficient as possible.

Julia also offers several other performance-oriented utilities. One of them is the so-called dot syntax [2]. When calling a function over some collection, adding a dot at the end of its name (e.g., `sign.`) will call the function element-wise rather than on the whole

collection, benefitting from pre-implemented efficient loops. Additionally, chaining operations, as in `sin.(exp(...))`, allows Julia to fuse the loops. That is, it creates a single efficient loop applying `sin(exp(...))` to each element directly, without intermediate allocations for the result of `exp(...)`.

Further relevant features include macros and metaprogramming [2]. Macros are special functions that allow the programmer to alter the execution of a program in ways a standard function cannot. They are denoted by starting with the character `@`. An example of one such macro is `@threads`, which parallelizes the subsequent loop across multiple threads. On a similar note, Julia allows for simple C interoperability through the `@ccall` macro, enabling the programmer to leverage highly optimized codebases without having to reimplement them. Moreover, metaprogramming is performed through the `@generated` macro; with it functions generate expressions at compile time, to be later applied on runtime data. As an example, below is a function to sum the elements of an N -tuple, generating the summation expression at compile time and executing it at runtime:

```

1 @generated function sum(t::NTuple{N, Any}) where N
2     expr = :(0)
3
4     for i in 1:N
5         expr = :( $expr + t[$i] )
6     end
7
8     return expr
9 end

```

Finally, it is worthwhile to discuss Julia's unicode support [2]. By supporting unicode characters, Julia can be written using math-like expressions like no other programming language can. Consider the step size calculation in the conjugate gradient method, given by [23]:

$$\alpha = \frac{\mathbf{r} \cdot \mathbf{r}}{\mathbf{p} \cdot (\mathbf{A}\mathbf{p})},$$

$$\mathbf{x} = \mathbf{x} + \alpha\mathbf{p},$$

where $\mathbf{r}$, $\mathbf{p}$ and $\mathbf{x}$ are vectors, $\mathbf{A}$ is a matrix, and α is some step size. In Julia, this can be written as:

```

1 α = (r • r) / (p • (A * p))
2 x = x + α * p

```

In the snippet above we were able to use α directly instead of `alpha`, and `•` was overloaded to the dot product within the builtin `LinearAlgebra.jl` package.

Overall, the combination of these features makes Julia an ideal language to solve the two-language problem. Its math-like syntax and dynamic nature lower the barrier to entry for young researchers, and method specialization ensures the computational efficiency

required by electromagnetics FEM solvers is achieved. Furthermore, utilities such as the `@ccall` macro provide a simple way of interfacing with battle-tested Fortran or C code-bases, such as HOFEM. Following GREMA's recent work, this thesis leverages Julia's modern features to develop an efficient and accessible FEM framework.

3. IMPLEMENTATION

This section presents the Julia implementation of the method described in section 2.1. Rather than categorizing the code by files, the discussion follows the computational workflow of a typical program, explaining the architecture in the order of execution.

3.1. Elements

The first step in the execution pipeline is defining the fundamental building blocks of the discretized domain: the geometric elements. To establish an easily extensible architecture, all element types are built upon a base abstract type:

```
1 abstract type AbstractElement{T <: Real} end
```

This parametric design allows the user to choose the precision at which the solver runs.

While this abstract type is designed to allow for arbitrary element types, the scope of the thesis is limited to tetrahedral meshes. The concrete `Tetrahedron` type is given by:

```
1 struct Tetrahedron{T} <: AbstractElement{T}
2     vertices::SMatrix{3, 4, T, 12}
3 end
```

Since a tetrahedron is uniquely determined by its four vertices, that is all the data that must be stored. Specifically, a 3×4 matrix, where the i -th column contains the i -th vertex, is stored.

The type `SMatrix` from the `StaticArrays.jl` package [24] was chosen over Julia's native `Matrix` to hold the vertices due to its underlying memory management; Julia's `Matrix` stores the data on the heap, whilst `SMatrix` does so on the stack. For a structure such as `Tetrahedron` that will be created and destroyed repeatedly inside hot-loops, the cost of repeated heap allocation and deallocation is too high.

To provide a more intuitive API, Julia's `Base.getindex` function is implemented for `Tetrahedron`:

```
1 @inline function Base.getindex(tet::Tetrahedron{T}, vertex_index::Int) where T
2     @boundscheck checkbounds(1:4, vertex_index)
3
4     return tet.vertices[:, vertex_index]
5 end
```

This allows accessing the vertex i using `tet[i]` rather than `tet.vertices[:, i]`. Furthermore, it offers an extra layer of safety by using the `@boundscheck` macro, which can

be disabled in hot-loops using the complementary `@inbounds` macro.

For the code to maintain consistent traversal of geometric elements, which is crucial to maintain correct DoF ordering and orientation during assembly, each concrete type must define the canonical ordering of its edges and vertices. For Tetrahedron, these are given by:

```

1  const TETRAHEDRON_EDGE_VERTICES =
2      SVector{6, Tuple{Int, Int}}(
3          (1, 2), (2, 3), (3, 1), # Base edges
4          (1, 4), (2, 4), (3, 4) # Vertical edges
5      )
6
7  const TETRAHEDRON_FACE_VERTICES =
8      SVector{4, Tuple{Int, Int, Int}}(
9          (1, 2, 3), # Base face
10         (1, 3, 4), (1, 4, 2), (2, 4, 3) # Side faces
11     )

```

The tuples within each list define the canonical orientation of each topological entity of the element. For instance, the tuple (1, 2) defines the first local edge as being directed from vertex 1 to vertex 2.

Each element must also expose the default constructor to yield the reference element of the chosen geometry. The tetrahedral reference element is defined as:

```

1  function Tetrahedron{T}() where T
2      return Tetrahedron(SMatrix{3, 4, T, 12}([
3          zero(T)  one(T)  zero(T)  zero(T);
4          zero(T)  zero(T)  one(T)  zero(T);
5          zero(T)  zero(T)  zero(T)  one(T)
6      ]))
7  end

```

Some geometric utilities that each concrete element type must define are the edge tangents, face normals, and the Jacobian $\mathbf{J}_K$. The tangent function must receive an element and an edge tuple (typically extracted from the edges vector), and return its unnormalized tangent in the specified orientation as an SVector. For Tetrahedron:

```

1  function tangent(tet::Tetrahedron{T}, edge::Tuple{Int, Int}) where T
2      i, j = edge
3
4      return tet[j] - tet[i]
5  end

```

Similarly, the normal function must take an element and a face tuple (again, typically from the faces vector), and return its unnormalized normal as an SVector:

```

1 function normal(tet::Tetrahedron{T}, face::Tuple{Int, Int, Int}) where T
2     i, j, k = face
3
4     return cross(tet[j] - tet[i],
5                 tet[k] - tet[i])
6 end

```

Finally, the jacobian function must take an element and return the Jacobian of the transformation Φ_K , as described in section 2.1.2:

```

1 function jacobian(tet::Tetrahedron{T}) where T
2     return transpose(SMatrix{3, 3, T, 9}(hcat(
3         tet.vertices[:, 2] - tet.vertices[:, 1],
4         tet.vertices[:, 3] - tet.vertices[:, 1],
5         tet.vertices[:, 4] - tet.vertices[:, 1]
6     )))
7 end

```

3.2. Meshes

With the geometric elements defined, the next step is to generate and load the mesh containing the domain information.

3.2.1. Mesh Generation

Gmsh [18] is a “three-dimensional finite element mesh generator with built-in pre- and post-processing facilities” that is widely used in the industry due to it being open source, free, and including out-of-the-box CAD integration. This, and its native Julia interface, made it the obvious choice to generate and load meshes.

To use Gmsh in Julia, it must first be imported as `import Gmsh: gmsh`. Before calling any of the functions available, it must be initialized with `gmsh.initialize()`. Once finished, and before program exit, Gmsh must be deinitialized using `gmsh.finalize()`.

To make simple test meshes, the OpenCASCADE module `gmsh.model.occ` [18] offers several geometric primitives. For instance, `gmsh.model.occ.addBox` adds a box given two vertices. More complex constructs and transformations are offered, such as splines, circles and general spheres, or rotations and translations. After all the geometry is created, a synchronization step must be performed by calling `gmsh.model.occ.synchronize()`.

From the Julia interface, Gmsh’s generation and saving options can also be altered by calling `gmsh.option.setNumber(option, value)`, which sets the selected option to the given value. After the geometry has been set, and the generator has been configured, the mesh can be generated using `gmsh.model.mesh.generate(3)`, where the parameter

indicates the dimensions of the mesh. The final step is to save the generated mesh calling `gmsh.write(filename)`, with `filename` being the path where the mesh will be saved. As an example, the following function can be used to generate a unit cube with mesh size `lc`:

```

1  function create_box(filename::String="mesh.msh", lc::Float64=0.1)
2      gmsh.initialize()
3
4      # 1 × 1 × 1 cube
5      gmsh.model.occ.addBox(0.0, 0.0, 0.0, 1.0, 1.0, 1.0)
6      gmsh.model.occ.synchronize()
7
8      # Set mesh size for all points
9      gmsh.option.setNumber("Mesh.MeshSizeMin", lc)
10     gmsh.option.setNumber("Mesh.MeshSizeMax", lc)
11
12     # Use Delaunay algorithm for 3D meshing (to ensure all elements are tetrahedra)
13     gmsh.option.setNumber("Mesh.Algorithm3D", 1)
14
15     gmsh.model.mesh.generate(3)
16
17     gmsh.option.setNumber("Mesh.MshFileVersion", 2.2)
18     gmsh.option.setNumber("Mesh.Binary", 0)
19
20     gmsh.write(filename)
21
22     gmsh.finalize()
23 end

```

The generated mesh is saved using the `.msh` format. This format and Gmsh also have the ability to set entities to different material and boundary groups. In this thesis this information is not read from the mesh file. The entire domain is one material, and, since only Dirichlet boundary conditions are considered, boundaries are generated from the geometry when loading the mesh. Therefore, only a subset of this format is used, the structure of which is detailed in table 3.1.

3.2.2. Mesh Loading and Processing

Once the mesh is generated, the next step is to parse and transform it into a format the code can use. To this end, the domain is stored using an indexed connectivity list. That is, rather than parsing into a vector of fully-formed elements (which would duplicate node data), a global vector of nodes and a vector of elements defined by the indices of their vertices in the nodes vector are used. This not only saves memory, but also allows the solver to keep track of nodes shared by several elements. The following function uses Gmsh to read a tetrahedral mesh:

Table 3.1. THE .MSH FILE FORMAT [18].

Block Tags	Description	Typical Data
\$MeshFormat	Defines the Gmsh file version, file type (ASCII or binary), and floating-point data size.	Version, file-type, data-size (e.g., 2.2 0 8).
\$EndMeshFormat		
\$Nodes	Contains the mesh vertices. Groups nodes by the geometric entity they belong to, listing their tags and coordinates (x, y, z).	Entity dimensions/tags, total number of nodes, and coordinate arrays.
\$EndNodes		
\$Elements	Defines the finite elements (e.g., lines, triangles, tetrahedra). Elements are grouped by the geometric entity they discretize.	Element type (e.g., 2 for triangle, 4 for tetrahedron), tags, and node connectivity lists.
\$EndElements		

```

1 function read_gmsh_mesh(filename::String)
2     gmsh.initialize()
3     gmsh.open(filename)
4
5     # Get only tetrahedra (Type 4)
6     tetTags, tetNodeTags = gmsh.model.mesh.getElementsByType(4)
7
8     # Find unique nodes used by these tetrahedra
9     unique_node_tags = unique(tetNodeTags)
10
11     # Get coordinates for only those nodes
12     nodes = Vector{Vector{Float64}}{
13         undef, length(unique_node_tags)
14     }
15
16     tag_to_idx = Dict{Int, Int}{}
17     for (i, tag) in enumerate(unique_node_tags)
18         coord, _, _ = gmsh.model.mesh.getNode(tag)
19         nodes[i] = coord
20         tag_to_idx[tag] = i
21     end
22
23     # Build elements list
24     num_tets = length(tetTags)
25     elements = Vector{NTuple{4, Int}}{undef, num_tets}
26     for i in 1:num_tets
27         idx = 4 * (i - 1)
28         elements[i] = (

```

```

29         tag_to_idx[tetNodeTags[idx + 1]],
30         tag_to_idx[tetNodeTags[idx + 2]],
31         tag_to_idx[tetNodeTags[idx + 3]],
32         tag_to_idx[tetNodeTags[idx + 4]]
33     )
34 end
35
36 gmsh.finalize()
37
38 return nodes, elements
39 end

```

First, the given mesh file is loaded using Gmsh. With the mesh parsed, all of the tetrahedral elements can be extracted using `gmsh.model.mesh.getElementsByType(4)`, where 4 is the identifier for tetrahedra. This function returns `tetTags`, a vector containing a unique id for each element loaded, and `tetNodeTags`, a flat vector with the tag for each vertex of each element. That is, `tetNodeTags[4(i - 1) + 1..4i]` are the tags of the vertices of the i -th tetrahedron.

Next, the actual node data is obtained from Gmsh. A vector with the unique tags appearing in `tetNodeTags` is created and looped over, calling `gmsh.model.mesh.getNode` to obtain the coordinates of every node. The node gets stored in a pre-allocated vector and, using a dictionary, its new index in the vector is stored with the old tag as the key.

Finally, to create the elements vector, a loop over the total number of elements translates the tags in `tetNodeTags` to indices, storing each element as a 4-tuple in the vector. The function then returns `nodes`, a vector with all the nodes in the mesh, stored each as another vector, and `elements`, a vector with a 4-tuple of node indices for each tetrahedron.

With the mesh loaded in a suitable format, the next step is to generate the DoF mapping. In essence, this step assigns a global DoF index to each local DoF of an element, ensuring shared DoFs have the same global index while enforcing orientation. The function responsible for this is `generate_dof_mapping`. Due to how related this task is to detecting the boundary, the function is also tasked with it.

Before assigning global DoFs, a global sign convention must be established. The function tasked with this is:

```

1 function isigno(v::SVector{3, T}; tol=1e-12) where {T <: Real}
2     s = -1
3     if abs(v[1]) <= tol && abs(v[2]) <= tol && abs(v[3]) <= tol
4
5         s = 0
6     elseif v[1] > tol ||
7         (abs(v[1]) <= tol && v[2] > tol) ||
8         (abs(v[1]) <= tol && abs(v[2]) <= tol && v[3] > tol)
9
10        s = 1

```

```

11     end
12
13     return s
14 end

```

This function takes a vector, and returns the sign of its first non-zero entry. It is also necessary to define some types to aid in identifying the boundary:

```

1 struct BoundaryEdge
2     nodes::Tuple{Int, Int}
3     element_id::Int
4     dofs::Tuple{Int, Int}
5 end
6
7 struct BoundaryFace
8     nodes::Tuple{Int, Int, Int}
9     element_id::Int
10    dofs::Tuple{Int, Int}
11 end

```

The function `generate_dof_mapping` receives as input the nodes and elements vectors, as generated by `read_gmsh_mesh`. The first thing it does is pre-allocating some buffers:

```

1 # B[element_id, local_dof_index] ⇒ global_dof_index
2 B = zeros{Int, length(elements), 20} # 20 DoFs for  $\mathcal{K}_2$ 
3
4 # K: (global_node_id_1, global_node_id_2) ⇒ global_dof_index
5 global_edge_dofs = Dict{Tuple{Int, Int}, Int}{}
6 # K: (global_node_id_1, global_node_id_2, global_node_id_3) ⇒ global_dof_index
7 global_face_dofs = Dict{Tuple{Int, Int, Int}, Int}{}
8
9 # K: (global_node_id_1, global_node_id_2) ⇒ [element_ids...]
10 edge_to_elements = Dict{Tuple{Int, Int}, Vector{Int}}{}
11 # K: (global_node_id_1, global_node_id_2, global_node_id_3) ⇒ [element_ids...]
12 face_to_elements = Dict{Tuple{Int, Int, Int}, Vector{Int}}{}

```

The most important of these is B. It is a matrix that maps the local DoFs of an element to the corresponding global DoFs. Then, the function iterates through all elements and begins mapping edges and faces:

```

1 current_dof = 1
2
3 for (e, element) in enumerate(elements)
4     # Map Edges
5     for (i, edge) in enumerate(TETRAHEDRON_EDGE_VERTICES)
6         g1, g2 = element[edge[1]], element[edge[2]]
7         v1, v2 = SVector{3, Float64}(nodes[g1]), SVector{3, Float64}(nodes[g2])

```

```

8       $\tau = v_2 - v_1$ 
9      sign = isigno( $\tau$ )
10
11     if sign == -1
12          $g_1, g_2 = g_2, g_1$ 
13     end
14
15     # Such that isigno( $\tau$ ) is always +1
16     unique_edge =  $g_1, g_2$ 
17
18     if !haskey(global_edge_dofs, unique_edge)
19         global_edge_dofs[unique_edge] = current_dof
20         current_dof += 2
21     end
22
23     base_dof = global_edge_dofs[unique_edge]
24
25     if sign == 1
26         B[e, 2i - 1], B[e, 2i] = base_dof, base_dof + 1
27     else
28         B[e, 2i - 1], B[e, 2i] = base_dof + 1, base_dof
29     end
30
31     # Record that this element touches this edge
32     push!(get!(edge_to_elements, unique_edge, Int[]), e)
33 end
34
35 # Map Faces
36 for (i, face) in enumerate(TETRAHEDRON_FACE_VERTICES)
37      $g_1, g_2, g_3 = \text{sort}((\text{element}[\text{face}[1]], \text{element}[\text{face}[2]], \text{element}[\text{face}[3]]))$ 
38      $v_1, v_2, v_3 = \text{SVector}\{3, \text{Float64}\}(\text{nodes}[g_1]), \text{SVector}\{3, \text{Float64}\}(\text{nodes}[g_2]),$ 
39          $\hookrightarrow \text{SVector}\{3, \text{Float64}\}(\text{nodes}[g_3])$ 
40     n = cross( $v_2 - v_1, v_3 - v_1$ )
41     sign = isigno(n)
42
43     if sign == -1
44          $g_2, g_3 = g_3, g_2$ 
45     end
46
47     # Such that  $g_1 < \min(g_2, g_3)$  and isigno(n) is always +1
48     unique_face =  $g_1, g_2, g_3$ 
49
50     if !haskey(global_face_dofs, unique_face)
51         global_face_dofs[unique_face] = current_dof
52         current_dof += 2
53     end
54
55     base_dof = global_face_dofs[unique_face]
56
57     B[e, 12 + 2i - 1], B[e, 12 + 2i] = base_dof, base_dof + 1
58
59     # Record that this element touches this face

```

```

59     push!(get!(face_to_elements, unique_face, Int[]), e)
60     end
61 end

```

It begins by going through the element's edges. For each, it switches the vertices so that the tangent is globally positive, obtaining a unique tuple of indices identifying that edge. Next, it assigns a new DoF index, if one did not exist previously, and then assigns two DoFs to each edge, flipping if the sign is inverted. This flipping is necessary as if two edges touch with opposite signs, the first DoF of one will be the second of the other, as shown in figure 3.1. At the same time, the loop registers which elements touch which edges.

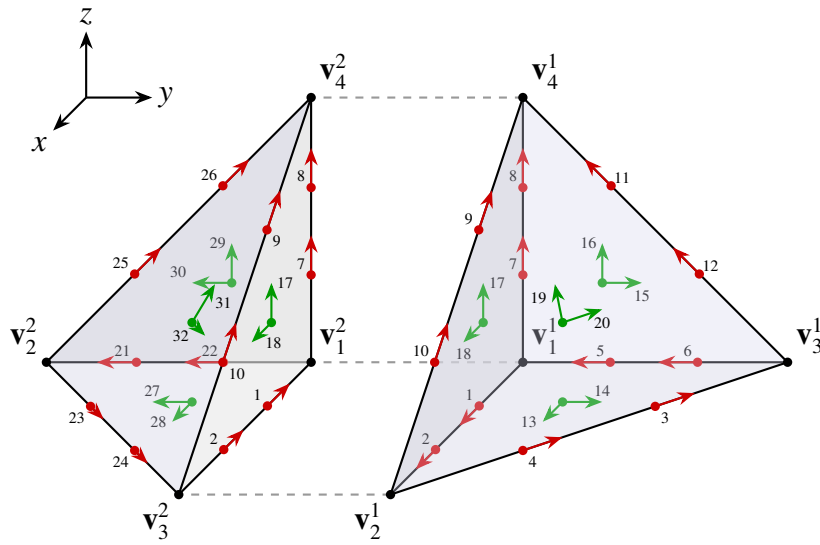


Fig. 3.1. Degrees of freedom for two tetrahedra sharing a face, numbered by `generate_dof_mapping`. Local edge DoFs are represented by red arrows, local face DoFs by green arrows, and global DoF indices by black numbers.

Once all the edges of an element are processed, the faces are considered. To obtain a global tuple identifying a face, the first index is fixed as the minimum of the three, and the other two are flipped depending on the normal's sign. Similarly to the edges, a new DoF index is chosen if none existed for this face, and the two DoFs are assigned. For the face DoFs no flipping is needed due to how the $\mathbf{q}$ polynomials are chosen in the DoF functionals. This will be explained in more detail in section 3.4. Here, the loop also registers the elements touching the face.

Since for second order Nédélec elements no volume DoFs are present, at this point the DoF mapping matrix B is complete, and only the identification of the boundary is left:

```

1 # Boundary Faces
2 boundary_faces_data = BoundaryFace[]
3 boundary_edge_keys = Set{Tuple{Int, Int}}()

```

```

4
5 for (face_key, elems) in face_to_elements
6     if length(elems) == 1
7         e = elems[1]
8         base_dof = global_face_dofs[face_key]
9
10        push!(boundary_faces_data, BoundaryFace(
11            face_key,
12            e,
13            (base_dof, base_dof + 1)
14        ))
15
16        # Register its edges as boundary edges
17        a, b, c = face_key
18        push!(boundary_edge_keys, (if isigno(SVector{3, Float64}(nodes[b] -
19            ↪ nodes[a])) == 1 (a, b) else (b, a) end))
20        push!(boundary_edge_keys, (if isigno(SVector{3, Float64}(nodes[c] -
21            ↪ nodes[a])) == 1 (a, c) else (c, a) end))
22        push!(boundary_edge_keys, (if isigno(SVector{3, Float64}(nodes[c] -
23            ↪ nodes[b])) == 1 (b, c) else (c, b) end))
24    elseif length(elems) > 2
25        error("Invalid mesh: face $(face_key) is shared by more than 2 elements.")
26    end
27end
28
29# Boundary Edges
30boundary_edges_data = BoundaryEdge[]
31for edge_key in boundary_edge_keys
32    e = edge_to_elements[edge_key][1] # Get one of the elements that touches this
33    ↪ edge
34    base_dof = global_edge_dofs[edge_key]
35
36    push!(boundary_edges_data, BoundaryEdge(
37        edge_key,
38        e,
39        (base_dof, base_dof + 1) # Always in order as we made sure nodes are ordered
40        ↪ so isigno( $\tau$ ) is +1 when creating global_edge_dofs
41    ))
42end
43
44dirichlet_dofs = Int[]
45for edge in boundary_edges_data
46    push!(dirichlet_dofs, edge.dofs[1], edge.dofs[2])
47end
48for face in boundary_faces_data
49    push!(dirichlet_dofs, face.dofs[1], face.dofs[2])
50end
51sort!(unique!(dirichlet_dofs))

```

To identify the boundary faces, it is sufficient to check which faces are only touched by one element. For each of these, their edges are also part of the boundary and must be registered,

making sure to use the unique edge tuple. Next, boundary edge data is obtained from the `base_dof` dictionary and inserted into the vector. Finally, the data vectors are condensed into a vector holding the global DoF indices of the boundary.

The function `generate_dof_mapping` returns:

- `B`: the DoF mapping matrix,
- `NDoF`: the number of global DoFs,
- `NEdge`: the number of edges,
- `dirichlet_dofs`: a vector of the global DoFs that sit on the boundary,
- `boundary_edges_data`: the data of the boundary edges, and
- `boundary_faces_data`: the data of the boundary faces.

3.3. Numerical Integration

Many operations in the FEM require performing some sort of integration. However, solving these integrals symbolically is unfeasible, and finding an exact solution is (almost) always impossible. Numerical integration is a set of algorithms used to numerically approximate integrals [25]. More precisely, in numerical integration a definite integral is approximated as:

$$\int_a^b f(x) dx \approx \sum_{i=1}^n w_i f(x_i),$$

where the x_i are the evaluation nodes, and w_i are the associated weights. In our application three different integration types are required:

- 1-dimensional integration: used in edge DoFs,
- 2-dimensional integration: for face DoFs, and
- 3-dimensional integration: used for any integral over the domain Ω , discretized into integrals of elements K .

3.3.1. 1-dimensional Integration

Every 1-dimensional integral over a segment e in 3-dimensional space, delimited by $\mathbf{v}_i$ and $\mathbf{v}_j$, can be computed using the standard 1-dimensional integral in the interval $[0, 1]$ by:

$$\int_e g dl = \int_0^1 g(\Phi_e(\hat{x})) \left\| \frac{\partial \Phi_e}{\partial \hat{x}} \right\|_2 d\hat{x},$$

where $\Phi_e : [0, 1] \rightarrow e$ is the transformation given by $\Phi_e(\hat{x}) = (1 - \hat{x}) \mathbf{v}_i + \hat{x} \mathbf{v}_j$, and $\|\partial\Phi_e/\partial\hat{x}\|_2 = \|\mathbf{v}_j - \mathbf{v}_i\|_2$ is the norm of its Jacobian. Then, for the numerical integral Gauss-Legendre quadrature [25] is chosen, which given an order n , takes the evaluation nodes x_i to be the roots of the Legendre polynomial P_n , defined as:

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n,$$

and the weights are:

$$w_i = \frac{2}{(1 - x_i^2) \frac{dP_n}{dx}(x_i)^2}.$$

This quadrature rule is defined over the interval $[-1, 1]$; the nodes and weights are translated to the interval $[0, 1]$ using the identities [25]:

$$x'_i = \frac{x_i + 1}{2}, \quad w'_i = \frac{w_i}{2},$$

such that the final approximation is:

$$\int_e g \, dl \approx \left\| \frac{\partial\Phi_e}{\partial\hat{x}} \right\|_2 \sum_{i=1}^n w'_i g(\Phi_e(x'_i)).$$

This approximation is exact when g is a polynomial of degree $2n - 1$ or less [25].

This rule is an obvious choice as Julia's `FastGaussQuadrature.jl` package [26] is available. This package offers the function `gausslegendre([T=Float64,] n::Integer) → x, w`, which returns the evaluation nodes and weights for order n from a precomputed table for small n , or dynamically computing them for larger orders. To avoid calling this function repeatedly, a global dictionary of nodes and weights is created, to which the tables of previously unseen orders are appended:

```

1 const _gausslegendre_cache = Dict{Int, Tuple{Vector{Float64}, Vector{Float64}}}()
2
3 function _gausslegendre(n::Int)
4     get!(_gausslegendre_cache, n) do
5         gausslegendre(n)
6     end
7 end

```

Then, the 1-dimensional numerical integration function is:

```

1 @inline function I(f::F, order::Int, ::Type{T}) where {F,T}
2     x, w = _gausslegendre(order)
3
4     acc = zero(T)
5
6     @inbounds for i in eachindex(x)
7         t = (x[i] + one(real(T))) / 2
8         wt = w[i] / 2

```

```

9
10     acc += wt * f(t)
11 end
12
13     return acc
14 end

```

3.3.2. 2-dimensional Integration

Similar to 1-dimensional integrals, every 2-dimensional integral over a surface f in 3-dimensional space (delimited by $\mathbf{v}_i$, $\mathbf{v}_j$ and $\mathbf{v}_k$) can be computed using the standard 2-dimensional integral over the right unit triangle, $\hat{\Delta}$, defined by the vertices $(0, 0)^\top$, $(1, 0)^\top$ and $(0, 1)^\top$:

$$\int_f g \, dS = \int_0^1 \int_0^{1-\hat{x}} g(\Phi_f(\hat{x}, \hat{y})) \left\| \frac{\partial \Phi_f}{\partial \hat{x}} \times \frac{\partial \Phi_f}{\partial \hat{y}} \right\|_2 \, d\hat{y} \, d\hat{x},$$

where $\partial \Phi_f : \hat{\Delta} \rightarrow f$ is the transformation given by $\Phi_f(\hat{x}, \hat{y}) = \mathbf{v}_i + \hat{x}(\mathbf{v}_j - \mathbf{v}_i) + \hat{y}(\mathbf{v}_k - \mathbf{v}_i)$, and $\|\partial \Phi_f / \partial \hat{x} \times \partial \Phi_f / \partial \hat{y}\|_2 = \|(\mathbf{v}_j - \mathbf{v}_i) \times (\mathbf{v}_k - \mathbf{v}_i)\|_2$ is the norm of its Jacobian.

For the numerical integral, a 2-dimensional Gauss-Legendre quadrature projected to the $[-1, 1] \times [-1, 1]$ cube via tensor products [25] is used. This rule is then transformed to the right unit triangle using the Duffy transformation [27], which gives the final approximation:

$$\int_f g \, dS \approx \left\| \frac{\partial \Phi_f}{\partial \hat{x}} \times \frac{\partial \Phi_f}{\partial \hat{y}} \right\|_2 \sum_{i=1}^n \sum_{j=1}^n w'_i w'_j (1 - x'_i) g(\Phi_f(x'_i, x'_j (1 - x'_i))).$$

This approximation is exact when g is a polynomial of degree $2n - 2$ or less [25], [27].

Then, the 2-dimensional numerical integration function is:

```

1  @inline function J(f::F, order::Int, ::Type{T}) where {F,T}
2      x, w = _gausslegendre(order)
3
4      acc = zero(T)
5
6      @inbounds for i in eachindex(x)
7          u = (x[i] + one(real(T))) / 2
8          wu = w[i] / 2
9
10         for j in eachindex(x)
11             v = (x[j] + one(real(T))) / 2
12             wv = w[j] / 2
13
14             x̂ = u
15             ŷ = (one(real(T)) - u) * v
16             detJ = one(real(T)) - u
17

```

```

18         acc += wu * wv * detJ * f(x̂, ŷ)
19     end
20 end
21
22     return acc
23 end

```

The reason to use this “extrapolated” rule is to easily allow for p -adaptivity (see section 3.3.4). Since the Gauss-Legendre rule for arbitrary orders can be trivially obtained, transforming it programmatically to the domain allows the application of arbitrary orders on it. However, if a specialized rule was hardcoded, the maximum order would be capped. The main caveat of this approach is the waste of some resources; on the tip of the triangle points are “squished” together, yielding an unnecessarily high node density, whilst a specialized rule would only use the necessary points, evenly spaced out through the entire triangle.

3.3.3. 3-dimensional Integration

Following the theory described in section 2.1.2, a 3-dimensional integral over an element K can be transformed into an integral over the reference element $\hat{K}$ by:

$$\int_K g \, dV = \int_{\hat{K}} g \circ \Phi_K \det(\mathbf{J}_K) \, d\hat{V} = \int_0^1 \int_0^{1-\hat{x}} \int_0^{1-\hat{x}-\hat{y}} g(\Phi_K(\hat{x}, \hat{y}, \hat{z})) \det(\mathbf{J}_K) \, d\hat{z} \, d\hat{y} \, d\hat{x}.$$

As for 2-dimensional integrals, to allow for simple p -adaptivity a 3-dimensional Gauss-Legendre quadrature over the unit cube [25] is used, transformed to the reference tetrahedron with the Duffy transformation [27] and giving the approximation:

$$\int_K g \, dV \approx \det(\mathbf{J}_K) \sum_{i=0}^n \sum_{j=0}^n \sum_{k=0}^n w'_i w'_j w'_k (1 - x'_i)^2 (1 - x'_j) g(\Phi_K(x'_i, x'_j (1 - x'_i), x'_k (1 - x'_i) (1 - x'_j))),$$

which is exact for polynomials of degree $2n - 3$ or less [25], [27].

Then, the 3-dimensional numerical integration function is:

```

1  @inline function III(f::F, order::Int, ::Type{T}) where {F,T}
2      x, w = _gausslegendre(order)
3
4      acc = zero(T)
5
6      @inbounds for i in eachindex(x)
7          u = (x[i] + one(real(T))) / 2
8          wu = w[i] / 2
9
10         for j in eachindex(x)
11             v = (x[j] + one(real(T))) / 2
12             wv = w[j] / 2

```

```

13
14     for k in eachindex(x)
15         w_node = (x[k] + one(real(T))) / 2
16         ww      = w[k] / 2
17
18          $\hat{x}$  = u
19          $\hat{y}$  = (one(real(T)) - u) * v
20          $\hat{z}$  = (one(real(T)) - u) * (one(real(T)) - v) * w_node
21
22         detJ = (one(real(T)) - u)^2 * (one(real(T)) - v)
23
24         acc += wu * wv * ww * detJ * f( $\hat{x}$ ,  $\hat{y}$ ,  $\hat{z}$ )
25     end
26 end
27
28
29     return acc
30 end

```

The same disadvantages exist for this rule as for the 2-dimensional one.

3.3.4. p -adaptivity

The quadrature rules described above allow for exact integration of polynomial functions, but for arbitrary functions choosing an integration order beforehand might yield errors much higher than desired. To solve this issue adaptivity is employed, of which there are two types [28]:

- h -adaptivity: the domain is subdivided until performing the integral on each subdivision yields no extra precision, and
- p -adaptivity: the order is increased until it gives no further precision.

In this thesis p -adaptivity was chosen as it is much simpler to implement. This choice is the reason why the Gauss-Legendre rule was selected for all geometries. For the numerical integration functions, p -adaptivity is implemented as follows:

```

1  function  $\int\!\!\int\!\!\int$ (f::F, tol::Real, ::Type{T}) where {F, T}
2      order = 1
3      prev2 =  $\int\!\!\int\!\!\int$ (f, order, T)
4      order += 1
5      prev1 =  $\int\!\!\int\!\!\int$ (f, order, T)
6
7      while true
8          order += 1
9          curr =  $\int\!\!\int\!\!\int$ (f, order, T)
10

```

```

11         if abs( curr - prev1) ≤ tol*(1+abs(curr)) &&
12            abs(prev1 - prev2) ≤ tol*(1+abs(prev1))
13             return curr
14         end
15
16         prev2, prev1 = prev1, curr
17     end
18 end

```

3.4. Computation of the Basis Functions

Hardcoding polynomial basis functions is standard for traditional FEM software. Instead, this Julia implementation adopts an algebraic approach by representing the basis functions as linear combinations of a standard three-dimensional monomial basis, and computing their coefficients systematically, applying the degrees of freedom functionals.

In order to make this architecture possible, some basic functionality is required to manage polynomials and their bases. To allow for future extensibility across different spaces, a base abstract type is defined:

```

1 abstract type AbstractBasis{Element <: AbstractElement, Order, T <: Real} end

```

To represent any polynomial space, it must first be possible to determine the number of terms that a monomial expansion of a polynomial of some order would yield:

```

1 @inline function num_monomials(::Val{Order}) where {Order}
2     # (d + 3) choose 3 = (d + 3)! / (3! * d!) = (d + 1)(d + 2)(d + 3) / 6
3     return (Order + 1) * (Order + 2) * (Order + 3) ÷ 6
4 end

```

With the size of the space known, the actual monomial expansion must be generated and evaluated at a specific point $\mathbf{x}$. Because basis evaluation occurs at every integration point within the innermost loops of the solver, performance is critical. Julia's metaprogramming capabilities via the `@generated` macro are the best choice to unroll the exponentiation loops at compile-time:

```

1 @generated function (::MonomialBasis{Order, T})(x::SVector{3, T}) where {Order, T}
2     exs = Expr[]
3
4     for d in 0:Order, i in d:-1:0, j in (d-i):-1:0
5         k = d - i - j
6
7         # Avoid powers of zero
8         if i==0 && j==0 && k==0

```

```

9         push!(exps, :(one(T)))
10      elseif i==0 && j==0
11         push!(exps, :(x[3]^$k))
12      elseif i==0 && k==0
13         push!(exps, :(x[2]^$j))
14      elseif j==0 && k==0
15         push!(exps, :(x[1]^$i))
16      else
17         push!(exps, :(x[1]^$i * x[2]^$j * x[3]^$k))
18      end
19  end
20
21  N = length(exps)
22  return :(SVector{$N, T}($(exps...)))
23 end

```

This generates the monomial expansion expression at compile-time, evaluating it at run-time at the given point.

To easily utilize these monomial expansions, a `Polynomial` type is also offered:

```

1 struct Polynomial{Order, T, NTerms}
2     coeffs::SMatrix{3, NTerms, T}
3 end
4
5 @inline function (p::Polynomial{Order, T, NTerms})(x::SVector{3, T}) where {Order, T,
6     ↪ NTerms}
7     m = MonomialBasis{Order, T}()(x)
8     return p.coeffs * m
9 end

```

The inline function under the type definition allows a polynomial `p` to be evaluated at a point `x` using the `p(x)` syntax.

Finally, since solving the variational problem requires taking curls of the basis functions, a way to get the derivatives of polynomials is needed:

```

1 @generated function diff_matrix(::Val{Order}, ::Val{dim}) where {Order, dim}
2     in_monos = NTuple{3, Int}[]
3     for d in 0:Order, i in d:-1:0, j in (d - i):-1:0
4         push!(in_monos, (i, j, d - i - j))
5     end
6
7     out_monos = NTuple{3, Int}[]
8     for d in 0:max(0, Order - 1), i in d:-1:0, j in (d - i):-1:0
9         push!(out_monos, (i, j, d - i - j))
10    end
11
12    Nin = length(in_monos)
13    Nout = length(out_monos)
14    out_idx = Dict{m ⇒ k for (k, m) in enumerate(out_monos)}

```

```

15
16   mat = zeros(Float64, Nout, Nin)
17   for (col, mono) in enumerate(in_monos)
18       p = mono[dim]
19       p == 0 && continue
20       new_mono = ntuple(k -> k == dim ? mono[k] - 1 : mono[k], 3)
21       row = get(out_idx, new_mono, 0)
22       row > 0 && (mat[row, col] = Float64(p))
23   end
24
25   vals = tuple(mat...)
26   return :(SMatrix{$Nout, $Nin, Float64, $(Nout * Nin)}$(vals...))
27 end

```

As before, this function uses the `@generated` macro to generate an expansion at compile time. In this case, it generates a differentiation matrix, i.e., a matrix that multiplied by the coefficients of a polynomial yields the coefficients of its derivative in some variable, denoted as `Dim`.

With the basic polynomial utilities defined, the specifics of the polynomial space of choice (first kind Nédélec) are considered. This space's dimension is smaller than that of a full polynomial space of the same order. The following function computes it for `Tetrahedron`:

```

1 @inline nedelec_dofs(::Type{<: Tetrahedron}, ::Val{Order}) where {Order} =
2   Order * (Order + 2) * (Order + 3) ÷ 2

```

Then, a Nédélec basis is represented using the concrete type:

```

1 struct Nedelec{Element, Order, T, NFuncs, NTerms} <: AbstractBasis{Element, Order, T}
2   coeffs::SArray{Tuple{3, NTerms, NFuncs}, T, 3}
3 end

```

Before delving into the computation of the basis, as with `Tetrahedron`, the `Base.getindex` function is overloaded to allow easy access to each of the polynomials of the basis:

```

1 function Base.getindex(basis::Nedelec{Element, Order, T, NFuncs, NTerms}, i::Int)
2   ↪ where {Element, Order, T, NFuncs, NTerms}
3   @boundscheck checkbounds(1:NFuncs, i)
4
5   return Polynomial{Order, T, NTerms}(basis.coeffs[:, :, i])
6 end

```

Then, a Nédélec basis is constructed for each element in the mesh using the constructor:

```

1 function Nedelec(element::Element, global_verts::SVector{4, Int}, ::Val{Order}) where
  ↳ {Element <: AbstractElement, Order}
2   T = eltype(element)
3   NFuncs = nedelec_dofs(Element, Val(Order))
4   NTerms = num_monomials(Val(Order))
5
6   return Nedelec{Element, Order, T, NFuncs, NTerms}({
7     SArray{Tuple{3, NTerms, NFuncs}, T}(compute_nedelec_coeffs(element,
8       ↳ global_verts, Val(Order)))
9   })
end

```

Each basis polynomial is created on the reference element, but adapted to each real element to ensure face DoFs are orthogonal. The function `compute_nedelec_coeffs` must be implemented for each order and element type pair. In this thesis only second order Nédélec polynomials of the first kind are considered.

Every polynomial $\mathbf{p}$ in $\mathcal{N}_2$ can be represented as [29]:

$$\mathbf{p} = \begin{bmatrix} a_1 + a_2x + a_3y + a_4z + Dy^2 - Fxy - Gxz + Hz^2 + Jyz \\ b_1 + b_2x + b_3y + b_4z - Dxy - Eyz + Fx^2 + Iz^2 - Jxz + Kxz \\ c_1 + c_2x + c_3y + c_4z + Ey^2 + Gx^2 - Hxz - Iyz - Kxy \end{bmatrix}.$$

Then, for all the polynomials in the basis, these coefficients can be organized into a matrix $\mathbf{C} \in \mathbb{R}^{20 \times 20}$:

$$\mathbf{C} = \begin{bmatrix} (a_1)_1 & (a_1)_2 & \cdots & (a_1)_{20} \\ (a_2)_1 & (a_2)_2 & \cdots & (a_2)_{20} \\ \vdots & \vdots & \ddots & \vdots \\ J_1 & J_2 & \cdots & J_{20} \\ K_1 & K_2 & \cdots & K_{20} \end{bmatrix}.$$

If Σ is chosen as the dual basis of $\mathcal{N}_2$, then it is required that $\mathbf{MC} = \mathbf{I}$, with $\mathbf{I}$ the identity matrix, and $\mathbf{M} \in \mathbb{R}^{20 \times 20}$ the matrix formed by applying the DoF functionals to the generating monomials:

$$\begin{aligned} a_1 &: (1, 0, 0)^\top, & a_2 &: (x, 0, 0)^\top, & a_3 &: (y, 0, 0)^\top, & a_4 &: (z, 0, 0)^\top, \\ b_1 &: (0, 1, 0)^\top, & b_2 &: (0, x, 0)^\top, & b_3 &: (0, y, 0)^\top, & b_4 &: (0, z, 0)^\top, \\ c_1 &: (0, 0, 1)^\top, & c_2 &: (0, 0, x)^\top, & c_3 &: (0, 0, y)^\top, & c_4 &: (0, 0, z)^\top, \\ D &: (y^2, -xy, 0)^\top, & E &: (0, -yz, y^2)^\top, & F &: (-xy, x^2, 0)^\top, & G &: (-xz, 0, x^2)^\top, \\ H &: (z^2, 0, -xz)^\top, & I &: (0, z^2, -yz)^\top, & J &: (yz, -xz, 0)^\top, & K &: (0, xz, -xy)^\top, \end{aligned}$$

i.e., $\mathbf{M}_{i,j} = \ell_i(\mathbf{m}_j)$ for $\mathbf{m}_j$ one of the monomials above. This stems from the definition of a dual basis; if $\mathcal{N}_2$ is the polynomial space with basis $\{\mathbf{N}_j\}$ and Σ is its dual space with basis $\{\ell_i\}$, then $\ell_i(\mathbf{N}_j) = \delta_{i,j}$, with $\delta_{i,j}$ the Kronecker delta function [7], [30]. Solving the system $\mathbf{C} = \mathbf{M}^{-1}$ yields the basis coefficients.

The `compute_nedelec_coeffs` function does exactly that. First, it creates a reference tetrahedron and initializes the relevant variables:

```

1 Order = 2
2 NDoFs = nedelec_dofs(Tetrahedron, Val(Order))
3 NMonomials = num_monomials(Val(Order))
4
5 monomials = nedelec_monomials(Val(Order))
6
7  $\hat{K}$  = Tetrahedron{T}()
8
9 M = zeros(MMatrix{NDoFs, NDoFs, Float64})

```

Then it iterates through all the edges:

```

1 # Edge DoFs: 6 edges * 2 DoFs per edge = 12 DoFs
2 for (e, edge) in enumerate(TETRAHEDRON_EDGE_VERTICES)
3      $v_i$  =  $\hat{K}$  [edge[1]]
4      $v_j$  =  $\hat{K}$  [edge[2]]
5
6     # Define the parametrization of the edge  $x: [0, 1] \rightarrow e, \hat{x} \rightarrow x(\hat{x})$ 
7      $x(\hat{x}) = (1 - \hat{x}) * v_i + \hat{x} * v_j$ 
8
9     # Define the tangent vector to the edge and the Jacobian of 'x'
10     $\hat{t}$  = tangent( $\hat{K}$ , edge)
11     $\hat{L}$  = norm( $\hat{t}$ ) #  $\|dx/dt\| = \|v_j - v_i\|$ : the length of the edge
12     $\hat{\tau}$  =  $\hat{t} / \hat{L}$  # Unit tangent vector
13
14    # Define the linear polynomials  $q_1$  and  $q_2$  on the edge.
15    # We choose  $q_1$  to be the normalized distance from  $v_j$  and
16    #  $q_2$  to be the normalized distance from  $v_i$ , so that they
17    # are linearly independent and span the space of linear
18    # polynomials on the edge.
19     $\hat{q}_1(x) = \text{norm}(x - v_j) / \hat{L}$ 
20     $\hat{q}_2(x) = \text{norm}(x - v_i) / \hat{L}$ 
21
22    for (m, monomial) in enumerate(monomials)
23         $M[2 * e - 1, m]$  =  $g_i(\hat{\tau}, \text{monomial}, \hat{q}_1, \hat{L}, x, \text{Val}(2))$ 
24         $M[2 * e, m]$  =  $g_i(\hat{\tau}, \text{monomial}, \hat{q}_2, \hat{L}, x, \text{Val}(2))$ 
25    end
26 end

```

Here, g_i is the edge DoF $\ell_{\hat{e},q}(\mathbf{p}) = \int_{\hat{e}} (\mathbf{p} \cdot \mathbf{t}_{\hat{e}}) q \, dl, \forall q \in \mathbb{P}_{p-1}(\hat{e})$, with polynomials $q_1(\hat{\mathbf{x}}) = \|\mathbf{x} - \mathbf{v}_j\|_2 / \hat{L}$ and $q_2(\hat{\mathbf{x}}) = \|\mathbf{x} - \mathbf{v}_i\|_2 / \hat{L}$, implemented as:

```

1 function  $g_i(\tau::SVector\{3,T\}, u::U, q::Function, L::T, x::X, ::Val\{2\}, \text{int\_order}::I=2)$ 
2      $\hookrightarrow$  where {T, U, X, I}
3      $f(\hat{x}) = \text{transpose}(u(x(\hat{x}))) * \tau * q(x(\hat{x}))$ 
4     return  $\int(f, \text{int\_order}, \text{typeof}(f(0.0))) * L$ 
5 end

```

The integrand is a polynomial of degree at most 3, hence why `order = 2` is used, optionally allowing to pass a tolerance if `u` is not a polynomial. Once all edges are done, the

faces are iterated over:

```

1  # Face DoFs: 4 faces * 2 DoFs per face = 8 DoFs
2  for (f, face) in enumerate(TETRAHEDRON_FACE_VERTICES)
3      vi =  $\hat{K}$  [face[1]]
4      vj =  $\hat{K}$  [face[2]]
5      vk =  $\hat{K}$  [face[3]]
6
7      # Define the parametrization of the face  $x: \delta \rightarrow f, (\hat{x}, \hat{y}) \mapsto x(\hat{x}, \hat{y})$ 
8       $x(\hat{x}, \hat{y}) = v_i + \hat{x} * (v_j - v_i) + \hat{y} * (v_k - v_i)$ 
9
10     # Use the global vertex indices to determine  $q_1$  and  $q_2$ 
11     # in a consistent way across shared faces
12     face_global_ids =
13         SVector(
14             global_verts[face[1]],
15             global_verts[face[2]],
16             global_verts[face[3]]
17         )
18
19     # Sort them to find the minimum and middle global indices
20     perm = sortperm(face_global_ids)
21
22     # Extract the physical coordinates of the min and mid
23     # vertices
24     v_min = element[face[perm[1]]]
25     v_mid = element[face[perm[2]]]
26
27     #  $q_1$  points from the minimum global ID to the next
28     # smallest global ID
29     q1 = v_mid - v_min
30     q1 /= norm(q1)
31
32     #  $q_2$  is orthogonal to both  $n * \text{isigno}(n)$  (to ensure
33     # consistent orientation across shared faces) and  $q_1$ 
34     n = normalize(normal(element, face))
35     q2 = cross(n * isigno(n), q1)
36     q2 /= norm(q2)
37
38     # Define the normal vector to the face and the Jacobian
39      $\hat{n}$  = normal( $\hat{K}$ , face)
40      $\hat{A}$  = norm( $\hat{n}$ ) #  $||\partial x / \partial \hat{x} \times \partial x / \partial \hat{y}|| = ||n||$ : 2 times the area of the face
41      $\hat{n}$  /=  $\hat{A}$  # Unit normal vector
42
43     # Transform  $q_1$  and  $q_2$  to the reference face using the
44     # Jacobian of the affine map from the reference
45     # tetrahedron to the real tetrahedron
46     JK = jacobian(element)
47      $\hat{q}_1$  = JK * q1
48      $\hat{q}_2$  = JK * q2
49
50     for (m, monomial) in enumerate(monomials)
51         M[12 + 2 * f - 1, m] = gi( $\hat{n}$ , monomial,  $\hat{q}_1$ ,  $\hat{A}$ , x, Val(2))

```

```

52     M[12 + 2 * f,      m] = gi( $\hat{n}$ , monomial,  $\hat{q}_2$ ,  $\hat{A}$ , x, Val(2))
53     end
54 end

```

Here, g_i is the face DoF $\ell_{\hat{f},q}(\mathbf{p}) = \int_{\hat{f}} (\mathbf{p} \times \mathbf{n}_{\hat{f}}) \cdot \mathbf{q} \, dS, \forall \mathbf{q} \in (\mathbb{P}_{p-2})^2(\hat{f})$, with polynomials (vectors) q_1 and q_2 chosen to match across shared faces, be orthogonal, and not require any flipping of the global DoF indices, implemented as:

```

1  function gi(n::SVector{3, T}, u::U, q::SVector{3, T}, A::T, x::X, ::Val{2},
   ↪  int_order::I=2) where {T, U, X, I}
2      f( $\hat{x}$ ,  $\hat{y}$ ) = (transpose(n × u(x( $\hat{x}$ ,  $\hat{y}$ ))) * q)
3      return  $\mathbb{I}(f, \text{int\_order}, \text{typeof}(f(0.0, 0.0))) * A$ 
4  end

```

The integrand is a polynomial of degree at most 2, hence why order = 2 is used, again optionally allowing to pass a tolerance if u is not a polynomial. Finally, the matrix is inverted; the computed coefficients are cleaned of near zeroes and mapped to the final coefficients array:

```

1  C = inv(M) # Solve M * C = I
2
3  # Map the local coefficients to global coefficients
4  coeffs = zeros(MArray{Tuple{3, NMonomials, NDoFs}, T})
5  coeffs[1, 1, :] .= C[1, :]           # [ 1, 0, 0] -> a1
6  coeffs[2, 1, :] .= C[5, :]           # [ 0, 1, 0] -> b1
7  coeffs[3, 1, :] .= C[9, :]           # [ 0, 0, 1] -> c1
8  coeffs[1, 2, :] .= C[2, :]           # [ x, 0, 0] -> a2
9  coeffs[2, 2, :] .= C[6, :]           # [ 0, x, 0] -> b2
10 coeffs[3, 2, :] .= C[10, :]          # [ 0, 0, x] -> c2
11 coeffs[1, 3, :] .= C[3, :]           # [ y, 0, 0] -> a3
12 coeffs[2, 3, :] .= C[7, :]           # [ 0, y, 0] -> b3
13 coeffs[3, 3, :] .= C[11, :]          # [ 0, 0, y] -> c3
14 coeffs[1, 4, :] .= C[4, :]           # [ z, 0, 0] -> a4
15 coeffs[2, 4, :] .= C[8, :]           # [ 0, z, 0] -> b4
16 coeffs[3, 4, :] .= C[12, :]          # [ 0, 0, z] -> c4
17                                     # [x2, 0, 0] -> 0
18 coeffs[2, 5, :] .= C[15, :]          # [ 0, x2, 0] -> F
19 coeffs[3, 5, :] .= C[16, :]          # [ 0, 0, x2] -> G
20 coeffs[1, 6, :] .= -C[15, :]         # [xy, 0, 0] -> -F
21 coeffs[2, 6, :] .= -C[13, :]         # [ 0, xy, 0] -> -D
22 coeffs[3, 6, :] .= -C[20, :]         # [ 0, 0, xy] -> -K
23 coeffs[1, 7, :] .= -C[16, :]         # [xz, 0, 0] -> -G
24 coeffs[2, 7, :] .= -C[19, :] + C[20, :] # [ 0, xz, 0] -> -J + K
25 coeffs[3, 7, :] .= -C[17, :]         # [ 0, 0, xz] -> -H
26 coeffs[1, 8, :] .= C[13, :]          # [y2, 0, 0] -> D
27                                     # [ 0, y2, 0] -> 0
28 coeffs[3, 8, :] .= C[14, :]          # [ 0, 0, y2] -> E
29 coeffs[1, 9, :] .= C[19, :]          # [yz, 0, 0] -> J
30 coeffs[2, 9, :] .= -C[14, :]         # [ 0, yz, 0] -> -E

```

```

31 coeffs[3, 9, :] := -C[18, :]           # [ 0,  0, yz] -> -I
32 coeffs[1, 10, :] := C[17, :]          # [z^2,  0,  0] -> H
33 coeffs[2, 10, :] := C[18, :]          # [ 0, z^2,  0] -> I
34                                     # [ 0,  0, z^2] -> 0
35
36 # Filter out small values to avoid numerical issues
37 coeffs[abs.(coeffs) .< 1e-12] := zero(T)

```

Figure 3.2 shows two example basis functions computed on the reference element.

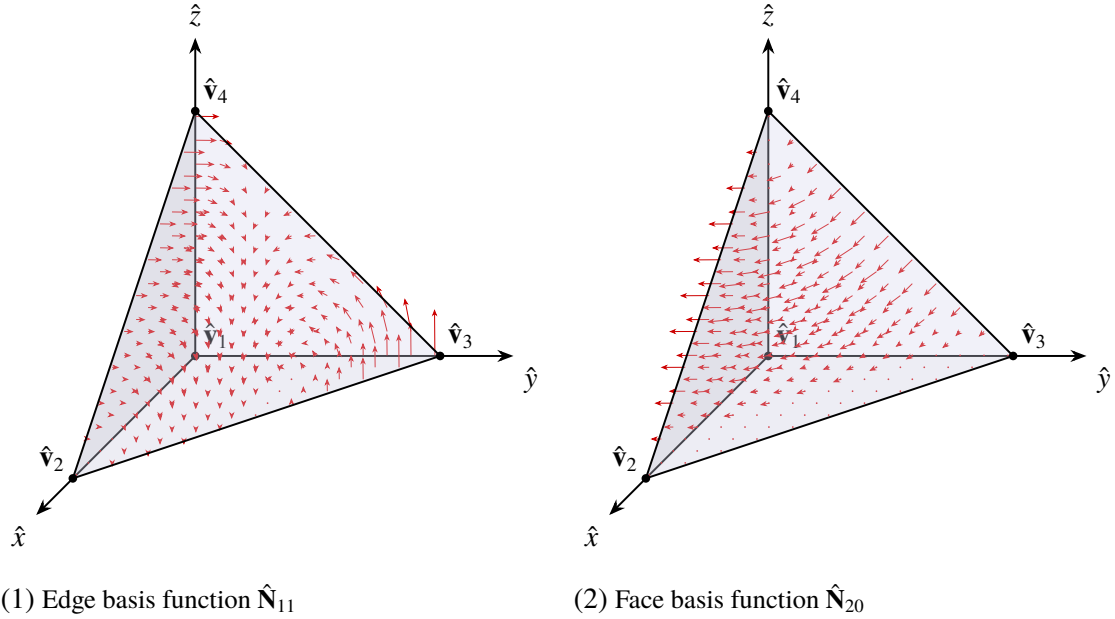


Fig. 3.2. Basis functions for $\hat{K}$ computed using `compute_nedelec_coeffs`.

With the Nédélec basis computed, to obtain its curl, i.e., the curl of all the basis polynomials, the following function is used:

```

1 function curl(N::Nedelec{Element, Order, T, NFuncs, NTerms}) where {Element, Order,
  ↪ T, NFuncs, NTerms}
2     Dx = diff_matrix(Val{Order}, Val{1})
3     Dy = diff_matrix(Val{Order}, Val{2})
4     Dz = diff_matrix(Val{Order}, Val{3})
5
6     NTerms_out = num_monomials(Val{Order - 1})
7     out = MArray{Tuple{3, NTerms_out, NFuncs}, T}(undef)
8
9     for f in 1:NFuncs
10         p1 = SVector{NTerms, T}(ntuple(i -> N.coeffs[1, i, f], Val{NTerms}()))
11         p2 = SVector{NTerms, T}(ntuple(i -> N.coeffs[2, i, f], Val{NTerms}()))
12         p3 = SVector{NTerms, T}(ntuple(i -> N.coeffs[3, i, f], Val{NTerms}()))
13
14         # curl = [∂p3/∂y - ∂p2/∂z, ∂p1/∂z - ∂p3/∂x, ∂p2/∂x - ∂p1/∂y]
15         c1 = Dy * p3 - Dz * p2
16         c2 = Dz * p1 - Dx * p3

```

```

17         c3 = Dx * p2 - Dy * p1
18
19         out[1, :, f] = c1
20         out[2, :, f] = c2
21         out[3, :, f] = c3
22     end
23
24     return Ndedelec{Element, Order-1, T, NFuncs, NTerms_out}(
25         SArray{Tuple{3, NTerms_out, NFuncs}, T}(out)
26     )
27 end

```

3.5. Assembly of the Global System

Theoretically, the assembly process involves computing the local matrices and adding their contributions to the global system. In practice rather than creating these matrices and then adding them to the global system, each local DoF of each element is iterated over, adding each entry directly to the global system.

Since the resulting system is sparse, all of these local entries are added to a manually-managed COO matrix, that is:

- a vector of row indices, `row_idx`,
- a vector of column indices, `col_idx`, and
- a vector of non-zero values, `data_M`.

As an example, to register a value of 5 at row 1 and column 2, we must append, at the same position in all three vectors, 1 to `row_idx`, 2 to `col_idx`, and 5 to `data_M`. Since the number of local DoFs per element and the total number of elements are known, these vectors can be preallocated to prevent reallocations in the hot-loop. The same happens with the right-hand side vector:

```

1 nelem = length(elements)
2 entries_per_element = 400 # 20 Nédélec DoFs per element
3 total_entries = nelem * entries_per_element
4
5 row_idx = Vector{Int}(undef, total_entries)
6 col_idx = Vector{Int}(undef, total_entries)
7 data_M = Vector{Float64}(undef, total_entries)
8
9 b = zeros(ComplexF64, NDoF)

```

Then, the assembly loop goes as follows: for every element index `e`, the element is extracted, reconstructed, and its relevant quantities are computed. That is:

```

1 element = elements[e]
2
3 K = Tetrahedron(SMatrix{3,4,Float64}([
4     nodes[element[1]];
5     nodes[element[2]];
6     nodes[element[3]];
7     nodes[element[4]]
8 ]))
9 J = jacobian(K)
10 invJ = inv(J)
11 detJ = det(J)
12
13  $\hat{\mathbf{N}}$  = Nedelec(K, SVector(element), Val(2))
14  $\nabla \times \hat{\mathbf{N}}$  = curl( $\hat{\mathbf{N}}$ )
15
16 signs = compute_signs(K)

```

`compute_signs` gives an array with the global sign of each edge tangent and face normal:

```

1 function compute_signs(element)
2     signs = ones{Int, 20}
3
4     # Edge orientations (DoFs 1 to 12)
5     for (i, edge) in enumerate(TETRAHEDRON_EDGE_VERTICES)
6         s = isigno(tangent(element, edge))
7
8         signs[2i - 1] = s
9         signs[2i] = s
10    end
11
12    # Face orientations (DoFs 13 to 20)
13    for (i, face) in enumerate(TETRAHEDRON_FACE_VERTICES)
14        s = isigno(normal(element, face))
15
16        signs[12 + 2i - 1] = s
17        signs[12 + 2i] = s
18    end
19
20    return signs
21 end

```

These signs are then used for shared edges or faces with opposite orientations, as they must be globally aligned. Then a 2-dimensional loop is run through each local DoF of the element, computing and assigning the entries:

```

1 offset = (e - 1) * entries_per_element
2 local_counter = 1
3
4 for i in 1:20

```

```

5     gi = B[e, i]
6
7     b[gi] -= im * k0 * η0 * compute_j(Ň[i], K[1], J, invJ, detJ, Jf) * signs[i]
8     b[gi] -= compute_m(Ň[i], K[1], J, invJ, detJ, ∇xinvμrMf) * signs[i]
9
10    for j in 1:20
11        gj = B[e, j]
12
13        idx = offset + local_counter
14        row_idx[idx] = gi
15        col_idx[idx] = gj
16
17        data_M[idx] = compute_K(∇xŇ[j], ∇xŇ[i], J, detJ) * signs[i] * signs[j]
18        data_M[idx] -= k0^2 * compute_M(Ň[j], Ň[i], invJ, detJ) * signs[i] *
19            ↪ signs[j]
20
21        local_counter += 1
22    end
end

```

Each of the `compute_` functions computes the entry of the respective matrix or vector from equation (2.12):

```

1 function compute_K(∇xŇi::Polynomial, ∇xŇj::Polynomial, J::SMatrix{3, 3, Float64},
2   ↪ j::Float64, invμr::SMatrix{3, 3, Float64}=SMatrix{3, 3, Float64}(I))
3   return ℑ((x̂, ŷ, ẑ) -> transpose(∇xŇj(SVector(x̂, ŷ, ẑ))) * (J * invμr *
4     ↪ transpose(J) * 1 / abs(j)) * ∇xŇi(SVector(x̂, ŷ, ẑ)), 3, Float64)
5 end
6
7 function compute_M(Ňi::Polynomial, Ňj::Polynomial, invJ::SMatrix{3, 3, Float64},
8   ↪ j::Float64, εr::SMatrix{3, 3, Float64}=SMatrix{3, 3, Float64}(I))
9   return ℑ((x̂, ŷ, ẑ) -> transpose(Ňj(SVector(x̂, ŷ, ẑ))) * (transpose(invJ) * εr *
10     ↪ invJ * abs(j)) * Ňi(SVector(x̂, ŷ, ẑ)), 4, Float64)
11 end
12
13 function compute_j(Ňi::Polynomial, v1::SVector{3, Float64}, J::SMatrix{3, 3,
14   ↪ Float64}, invJ::SMatrix{3, 3, Float64}, j::Float64, Jf::Function,
15   ↪ tol::Real=1e-12)
16   return ℑ((x̂, ŷ, ẑ) -> transpose(Ňi(SVector(x̂, ŷ, ẑ))) * (transpose(invJ) *
17     ↪ abs(j)) * Jf(v1 + transpose(J) * SVector(x̂, ŷ, ẑ)), tol, ComplexF64)
18 end
19
20 function compute_m(Ňi::Polynomial, v1::SVector{3, Float64}, J::SMatrix{3, 3,
21   ↪ Float64}, invJ::SMatrix{3, 3, Float64}, j::Float64, ∇xinvμrMf::Function,
22   ↪ tol::Real=1e-12)
23   return ℑ((x̂, ŷ, ẑ) -> transpose(Ňi(SVector(x̂, ŷ, ẑ))) * (transpose(invJ) *
24     ↪ abs(j)) * ∇xinvμrMf(v1 + transpose(J) * SVector(x̂, ŷ, ẑ)), tol, ComplexF64)
25 end

```

The integrals in `compute_K` and `compute_M` are exact, as their integrands are always polynomials of degree 2 and 4, respectively. In the case of `compute_j` and `compute_m`, the

passed terms may be arbitrary functions, thus using adaptive integration yields more precise results.

Once the loop finishes, $\mathbf{b}$ is complete and $\mathbf{M}$ must be compressed to CSC format using:

```
1 M = sparse(row_idx, col_idx, data_M, NDoF, NDoF)
```

Instead of keeping a triple for each non-zero entry with possible duplicates, the CSC format sums duplicate entries and “compresses” the coordinates, ending up with three smaller vectors:

- `data`, holding the deduplicated values,
- `indices`, storing the row index of each of the entries in `data`, and
- `indptr`, a vector indicating the start and end indices of each column in `indices`.

As an example, consider the matrix:

$$\begin{bmatrix} 1 & 0 & 0 & 5 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 7 \\ 0 & 4 & 0 & 6 & 0 \end{bmatrix}$$

In CSC format, it would be represented as:

- `data`: [1, 2, 3, 4, 5, 6, 7]
- `indices`: [1, 2, 3, 4, 1, 4, 3]
- `indptr`: [1, 2, 5, 5, 7, 8]

At this stage the system is technically ready to solve, but since $\mathbf{V}(\Omega)$ was chosen to make the boundary integrals vanish, the boundary conditions must now be enforced directly on the assembled matrix.

3.6. Imposition of the Dirichlet Boundary Conditions

The standard formulation of the Dirichlet boundary conditions:

$$\mathbf{n} \times \mathbf{E} = \mathbf{g}_D,$$

assumes $\mathbf{g}_D$ is always purely tangential on Γ_D . To ease this restriction, the more general projective form is implemented instead:

$$\mathbf{n} \times \mathbf{E} = \mathbf{n} \times \mathbf{g}_D,$$

where $\mathbf{g}_D$ is allowed to be a general vector field. This form is preferred from a computational perspective because it allows the solver to accept arbitrary analytical functions without requiring the user to manually pre-project them onto the tangent space of the boundary. This is particularly valuable for code verification using the Method of Manufactured Solutions (MMS), as explained later in section 4.2, where the exact analytical solution can be evaluated directly on the boundaries.

At the discrete level, this condition is naturally enforced through the geometric properties of the Nédélec space DoFs. For an edge e of the boundary Γ_D , its unit tangent vector $\mathbf{t}_e$ is orthogonal to the boundary normal $\mathbf{n}$, i.e., $\mathbf{t}_e \cdot \mathbf{n} = 0$. If $\mathbf{g}_D$ is decomposed into its tangential and normal components relative to the boundary, $\mathbf{g}_D = \mathbf{g}_{D,t} + g_{D,n}\mathbf{n}$, evaluating the functional yields:

$$\ell_{e,q}(\mathbf{g}_D) = \int_e (\mathbf{g}_{D,t} + g_{D,n}\mathbf{n}) \cdot \mathbf{t}_e q \, dl = \int_e (\mathbf{g}_{D,t} \cdot \mathbf{t}_e) q \, dl.$$

The dot product with the tangent essentially annihilates the normal component of $\mathbf{g}_D$. A similar filtering occurs with the face functionals and the cross product $\mathbf{g}_D \times \mathbf{n}_f$. Then, evaluating the DoF functionals with $\mathbf{g}_D$ yields the exact coefficients $g_i = \ell_i(\mathbf{g}_D)$ needed to reconstruct the tangential boundary projection $\mathbf{n} \times \mathbf{E}$. The following function computes these coefficients:

```

1  function compute_dirichlet_values(mesh_nodes, boundary_edges_data,
   ↪ boundary_faces_data, u)
2      g_values = Dict{Int, ComplexF64}()
3
4      # Edge Boundary Conditions
5      for b_edge in boundary_edges_data
6          v_i = SVector{3,Float64}(mesh_nodes[b_edge.nodes[1]])
7          v_j = SVector{3,Float64}(mesh_nodes[b_edge.nodes[2]])
8
9          x(ξ) = (1 - ξ) * v_i + ξ * v_j
10
11          τ = v_j - v_i # isigno(τ) is always +1
12          L = norm(τ)
13          τ = τ / L
14
15          q1(x) = norm(x - v_j) / L
16          q2(x) = norm(x - v_i) / L
17
18          g_values[b_edge.dofs[1]] = g_i(SVector{3,Float64}(τ), u, q1, L, x, Val(2),
   ↪ 1e-12)
19          g_values[b_edge.dofs[2]] = g_i(SVector{3,Float64}(τ), u, q2, L, x, Val(2),
   ↪ 1e-12)
20      end
21
22      # Face Boundary Conditions
23      for b_face in boundary_faces_data
24          v_i = SVector{3,Float64}(mesh_nodes[b_face.nodes[1]])
25          v_j = SVector{3,Float64}(mesh_nodes[b_face.nodes[2]])

```

```

26     v_k = SVector{3,Float64}(mesh_nodes[b_face.nodes[3]])
27
28     x( $\hat{x}$ ,  $\hat{y}$ ) = v_i +  $\hat{x}$  * (v_j - v_i) +  $\hat{y}$  * (v_k - v_i)
29
30     n = cross(v_j - v_i, v_k - v_i) # isigno(n) is always +1
31     A = norm(n)
32     n /= A
33
34     v_min = v_i
35     v_mid = if b_face.nodes[2] < b_face.nodes[3] v_j else v_k end
36
37     q_1 = v_mid - v_min
38     q_1 /= norm(q_1)
39
40     q_2 = cross(n, q_1)
41     q_2 /= norm(q_2)
42
43     g_values[b_face.dofs[1]] = g_i(n, u, q_1, A, x, Val(2), 1e-12)
44     g_values[b_face.dofs[2]] = g_i(n, u, q_2, A, x, Val(2), 1e-12)
45 end
46
47 return g_values
48 end

```

As u is arbitrary, a tolerance must be passed to the g_i function in order to get an accurate integral.

With the Dirichlet values computed, the final step is to force the system in equation (2.12) to conform to them in the boundary DoFs. Assume a standard linear system $\mathbf{Ax} = \mathbf{b}$ given by:

$$\begin{bmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} & \mathbf{A}_{1,3} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} & \mathbf{A}_{2,3} \\ \mathbf{A}_{3,1} & \mathbf{A}_{3,2} & \mathbf{A}_{3,3} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}.$$

Suppose index $i = 2$ is part of the boundary and we want to force $\mathbf{x}_2 = g$. The known values in $\mathbf{A}$ must be subtracted from $\mathbf{b}$, the column and row i must be zeroed out, and the diagonal entry must be set to 1:

$$\begin{bmatrix} \mathbf{A}_{1,1} & 0 & \mathbf{A}_{1,3} \\ 0 & 1 & 0 \\ \mathbf{A}_{3,1} & 0 & \mathbf{A}_{3,3} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 - \mathbf{A}_{1,2}g \\ g \\ \mathbf{b}_3 - \mathbf{A}_{3,2}g \end{bmatrix}.$$

This is implemented as:

```

1 function apply_dirichlet_bc!(M::AbstractMatrix,
2                               b::AbstractVector,
3                               dirichlet_values::Dict{Int, <: Number})
4     for (d, g) in dirichlet_values
5         if g != zero{ComplexF64}()
6             b .-= M[:, d] * g

```

```

7     end
8 end
9
10 for (d, g) in dirichlet_values
11     M[d, :] .= 0.0
12     M[:, d] .= 0.0
13     M[d, d] = 1.0
14     b[d]    = g
15 end
16
17 dropzeros!(M)
18 return nothing
19 end

```

The matrix passed to this function is already in CSC format, making this extremely inefficient. Fortunately, these boundary values can be enforced in an “online” regime as the system is assembled (see section 3.7).

3.7. Parallelization with MPI

MPI [20] is a standard for writing message-passing programs, that is, multi-process programs that are able to communicate between their processes. To understand why this interface is necessary, it is important to note the two kinds of parallelism:

- **Thread parallelism:** a process spawns threads to divide work among them. The main benefit of threads is that they share the memory space of the spawning thread (the main process). However, they are locked to one machine.
- **Process parallelism:** several processes are spawned, again, to divide work between them. The main advantage over threads is that these processes can be started on different machines, allowing the creation a “web” of computers (cluster) that all contribute to a unified task. The primary drawback is that these processes share no memory inherently, hence the need for an interface for sending data between them.

Both of these models can be used in tandem: each of the processes spawned can use thread parallelism locally.

Using MPI is relatively simple. At its core, every MPI program must initialize the MPI library, create a communicator (responsible for sending and receiving messages), and finalize it before the program ends. Using Julia’s `MPI.jl` package [31] the simplest MPI program would be:

```

1 MPI.Init()
2
3 comm = MPI.COMM_WORLD

```

```

4 rank = MPI.Comm_rank(comm)
5 size = MPI.Comm_size(comm)
6
7 print("Hello world, I am rank $rank of $size\n")
8
9 MPI.Barrier(comm)

```

Then, an MPI program can be run through the `mpiexec` command, specifying the `-n` flag to the desired number of processes. Here, `MPI.COMM_WORLD` is the default communicator allowing the program to communicate with any process, `MPI.Comm_rank` and `MPI.Comm_size` give the process id and the total number of processes, respectively, and `MPI.Barrier` blocks the execution of all processes until they all reach this function. It is important to note that manually calling `MPI.Finalize()` is not necessary as the Julia interface automatically does so on program exit.

A few more MPI functions that will be useful for this thesis are:

- `MPI.bcast(obj, root, comm)`: distributes `obj` from process `root` to all other processes.
- `MPI.Reduce!(sendbuf, recvbuf, op, root, comm)`: reduces the `sendbuf`'s of all processes applying `op` in an element-wise manner into `recvbuf`. In our case, `op` will be `MPI.SUM`.
- `MPI.Gatherv!(sendbuf, recvbuf, counts, root, comm)`: gathers all `sendbuf`'s into `recvbuf` in rank order, each process sending `counts[ranks]` items.

With MPI introduced, the changes to the full program for parallelization purposes are now exposed. First, the mesh is loaded only in process 0 and broadcasted to all others:

```

1 # Read mesh and process it only on process 0, then broadcast to all other processes
2 if rank == 0
3     nodes, elements = read_gmsh_mesh(mesh_file)
4     B, NDoF, _, _, boundary_edges_data, boundary_faces_data =
5         ↪ generate_dof_mapping(nodes, elements)
6     dirichlet_values = compute_dirichlet_values(nodes, boundary_edges_data,
7         ↪ boundary_faces_data, V_mms)
8 else
9     nodes = nothing
10    elements = nothing
11    B = nothing
12    NDoF = nothing
13    boundary_edges_data = nothing
14    boundary_faces_data = nothing
15    dirichlet_values = nothing
16 end
17 nodes = MPI.bcast(nodes, 0, comm)

```

```

17 elements = MPI.bcast(elements, 0, comm)
18 B = MPI.bcast(B, 0, comm)
19 NDoF = MPI.bcast(NDoF, 0, comm)
20 boundary_edges_data = MPI.bcast(boundary_edges_data, 0, comm)
21 boundary_faces_data = MPI.bcast(boundary_faces_data, 0, comm)
22 dirichlet_values = MPI.bcast(dirichlet_values, 0, comm)

```

Each process is tasked with processing an equal amount of elements; each computes how many elements it must process and pre-initializes the local buffers:

```

1  # Each process gets approximately nelelem / size elements to work on
2  start_elem = (nelem * rank) ÷ size + 1
3  end_elem = (nelem * (rank + 1)) ÷ size
4
5  local_nelem = end_elem - start_elem + 1
6  total_local_entries = entries_per_element * local_nelem
7
8  row_idx = Vector{Int}(undef, total_local_entries)
9  col_idx = Vector{Int}(undef, total_local_entries)
10 data_M = Vector{Float64}(undef, total_local_entries)
11
12 b_local = zeros(ComplexF64, NDoF)

```

Before the assembly loop, it is important to highlight the main limitation regarding how the Dirichlet boundary conditions were enforced in section 3.6 (aside of its lack of speed). Since the final assembled matrix was accessed directly, this would involve running the full assembly loop, gathering the matrix in one process, and then enforcing the conditions in that same process while all others waste resources. This defeats the whole parallelization effort. For this, an “online” algorithm is used, enforcing them in each local matrix instead. Then, the assembly loop runs as before, but with the new algorithm:

```

1  for i in 1:20
2      gi = B[e, i]
3      is_dirichlet_gi = haskey(dirichlet_values, gi)
4
5      if !is_dirichlet_gi
6          b_local[gi] -= im * k_0 * η_0 * compute_j( $\hat{N}$ [i], K[1], J, invJ, detJ, Jf) *
              ↪ signs[i]
7          b_local[gi] -= compute_m( $\hat{N}$ [i], K[1], J, invJ, detJ,  $\nabla x \text{inv} \mu_r M_f$ ) * signs[i]
8      end
9
10     for j in 1:20
11         gj = B[e, j]
12         is_dirichlet_gj = haskey(dirichlet_values, gj)
13
14         idx = offset + local_counter
15         row_idx[idx] = gi
16         col_idx[idx] = gj
17     end
18 end

```

```

18         if is_dirichlet_gi
19             data_M[idx] = 0.0
20         else
21             data_M[idx] = compute_K( $\nabla \times \hat{N}$  [j],  $\nabla \times \hat{N}$  [i], J, detJ) * signs[i] * signs[j]
22             data_M[idx] -= k02 * compute_M( $\hat{N}$  [j],  $\hat{N}$  [i], invJ, detJ) * signs[i] *
                ↪ signs[j]
23
24             if is_dirichlet_gj
25                 b_local[gi] -= data_M[idx] * dirichlet_values[gj]
26                 data_M[idx] = 0.0
27             end
28         end
29
30         local_counter += 1
31     end
32 end

```

In the code above, if the global row is in the boundary, it is skipped entirely. If, however, the column is in the boundary, the contributions to the right hand side vector and the matrix entry are computed and subtracted, leaving the matrix entry as 0. Otherwise, the contributions are computed normally.

This algorithm also leaves the diagonal boundary entries in the matrix and the right-hand side vector as zero. This can easily be corrected by setting all boundary entries of b to the corresponding value, in every process, and adding the diagonal 1's to the matrix in just one process.

The final step is to pass the distributed system to a solver.

3.8. Solving the Linear System

The choice of solver is fully determined by the need to keep the system distributed. To this end, MUMPS [32] was chosen due to it natively being able to handle distributed sparse matrices, and having a community-maintained Julia interface.

Upon testing the interface, it became clear that the functionality allowing the sending of the sparse matrix directly was not yet implemented. This required instead to gather the system into the main process and send it from there, letting MUMPS distribute it again. Still, MUMPS was left as the solver of choice for parity with HOFEM.

4. VERIFICATION

To verify the correctness code, 4 meshes were generated by iteratively refining an initial coarse mesh. The code used to generate these meshes was:

```
1 function generate_refined_meshes(h_start::Float64, num_refinements::Int)
2     gmsh.initialize()
3
4     gmsh.model.occ.addBox(0.0, 0.0, 0.0, 1.0, 1.0, 1.0)
5     gmsh.model.occ.synchronize()
6
7     gmsh.option.setNumber("Mesh.MeshSizeMin", h_start)
8     gmsh.option.setNumber("Mesh.MeshSizeMax", h_start)
9     gmsh.option.setNumber("Mesh.Algorithm3D", 1)
10    gmsh.option.setNumber("Mesh.MshFileVersion", 2.2)
11    gmsh.option.setNumber("Mesh.Binary", 0)
12
13    gmsh.model.mesh.generate(3)
14
15    filename_initial = "meshes/box_ref_0.msh"
16    gmsh.write(filename_initial)
17    println("Generated initial mesh: $filename_initial (approx h = $h_start)")
18
19    current_h = h_start
20
21    for i in 1:num_refinements
22        gmsh.model.mesh.refine()
23
24        current_h /= 2.0
25        filename = "meshes/box_ref_$i.msh"
26
27        gmsh.write(filename)
28        println("Generated refined mesh: $filename (approx h = $current_h)")
29    end
30
31    gmsh.finalize()
32 end
33
34 generate_refined_meshes(1.0, 4)
```

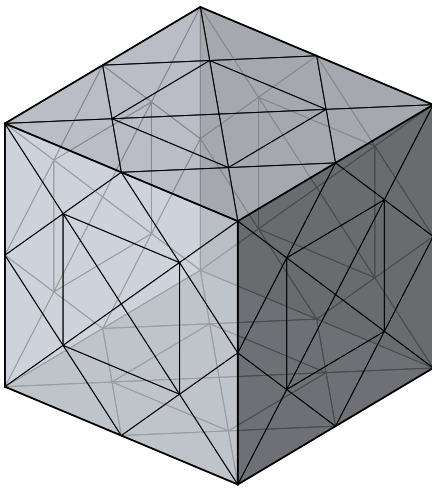
These meshes can be visualized in figure 4.1, along with their number of elements, number of DoFs, and computed refinement h . The refinement is computed using:

```
1 function compute_h_max(nodes, elements)
2     h_max = 0.0
3     for element in elements
4         for i in 1:3
5             for j in (i+1):4
```

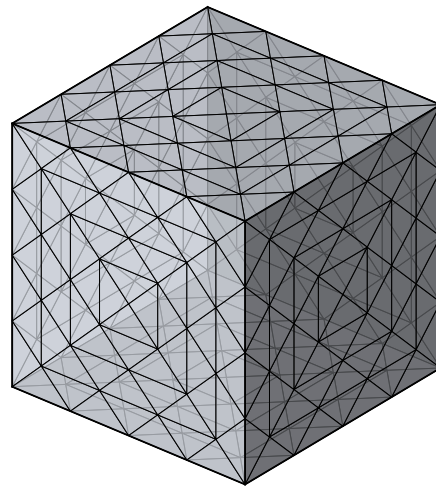
```

6         p1 = nodes[element[i]]
7         p2 = nodes[element[j]]
8
9         dist = norm(p1 - p2)
10        if dist > h_max
11            h_max = dist
12        end
13    end
14 end
15 end
16 return h_max
17 end

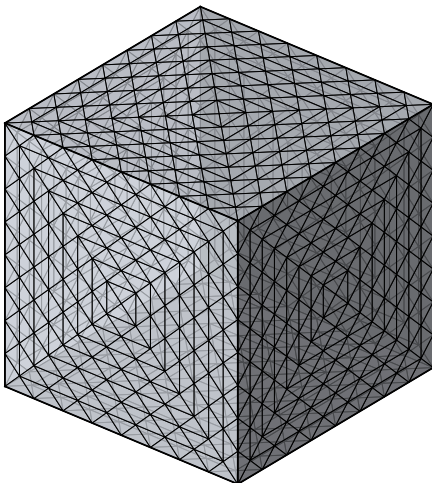
```



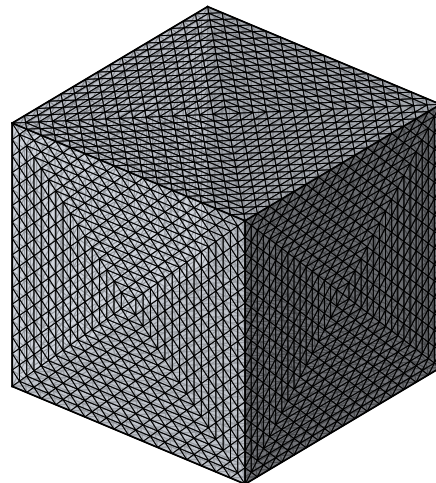
(1) 192 elements, 1468 DoFs, and $h \approx 0.6124$.



(2) 1536 elements, 10 712 DoFs, and $h \approx 0.3062$.



(3) 12 288 elements, 81 712 DoFs, and $h \approx 0.1531$.



(4) 98 304 elements, 638 048 DoFs, and $h \approx 0.0765$.

Fig. 4.1. Test meshes

4.1. Matrix Properties Test

The stiffness and mass matrices defined in section 2.1.4 must present some mathematical properties, which must be verifiable with the implementation described in section 3.

4.1.1. Mass Matrix

The mass matrix, whose entries are given by:

$$\mathbf{M}_{i,j} = \int_{\Omega} \mathbf{N}_j \cdot (\boldsymbol{\epsilon}_r \mathbf{N}_i) dV,$$

must be symmetric. This can be seen from the fact that, trivially:

$$\mathbf{N}_j \cdot (\boldsymbol{\epsilon}_r \mathbf{N}_i) = \mathbf{N}_i \cdot (\boldsymbol{\epsilon}_r \mathbf{N}_j),$$

if $\boldsymbol{\epsilon}_r = \boldsymbol{\epsilon}_r^T$, which is the case for all reciprocal media [9], [10], [12]. In Julia, the symmetry of $\mathbf{M}$ can be checked using the following code:

```
1 isapprox(M, transpose(M), rtol=1e-12)
```

$\mathbf{M}$ must also be positive definite. For a complex Hermitian matrix $\mathbf{A}$, this implies that $\mathbf{g}^H \mathbf{A} \mathbf{g} > 0$ for any nonzero complex vector $\mathbf{g}$ [33]. This condition is equivalent to all eigenvalues λ_i of $\mathbf{A}$ being strictly positive. Since $\mathbf{M}$ is symmetric rather than Hermitian, this condition varies slightly; every complex symmetric matrix can be decomposed into a sum of two Hermitian matrices:

$$\mathbf{M} = \mathbf{B} + i\mathbf{C},$$

where $\mathbf{B}$ and $\mathbf{C}$ are called the Hermitian and skew-Hermitian parts of $\mathbf{M}$, respectively, and are given by:

$$\mathbf{B} = \frac{\mathbf{M} + \mathbf{M}^H}{2}, \quad \mathbf{C} = \frac{\mathbf{M} - \mathbf{M}^H}{2i}.$$

Since $\mathbf{M}$ is complex symmetric, $\mathbf{M}^H = \overline{\mathbf{M}}$, giving $\mathbf{B} = \text{Re}\{\mathbf{M}\}$ and $\mathbf{C} = \text{Im}\{\mathbf{M}\}$.

As shown in [33], a complex symmetric matrix $\mathbf{M}$ is positive definite if its Hermitian part, $\text{Re}\{\mathbf{M}\}$, is positive definite. This requires:

$$\mathbf{g}^H \text{Re}\{\mathbf{M}\} \mathbf{g} = \text{Re}\{\mathbf{g}^H \mathbf{M} \mathbf{g}\} > 0,$$

which can be verified using:

```
1 eigs_M, _ = eigs(real(M); nev=1, which=:SR)
2 real(eigs_M[1]) > 1e-12
```

This test passed for all meshes, thus the mass matrix is correctly implemented.

For positive definiteness to hold, ϵ_r must also be positive definite; this is true for all standard passive media [9], [12].

To see why positive definiteness must hold, let us take a look at the expanded quadratic form for our discrete field $\mathbf{E}_h = \sum_{i=1}^N g_i \mathbf{N}_i$, defined in section 2.1.4:

$$\mathbf{g}^H \mathbf{M} \mathbf{g} = \int_{\Omega} \overline{\mathbf{E}}_h \cdot (\epsilon_r \mathbf{E}_h) dV.$$

Clearly, since the real part of ϵ_r is positive definite, we must have that $\text{Re}\{\mathbf{g}^H \mathbf{M} \mathbf{g}\} > 0$ for all nonzero fields.

4.1.2. Stiffness Matrix

Similarly, the stiffness matrix, whose entries are given by:

$$\mathbf{K}_{i,j} = \int_{\Omega} (\nabla \times \mathbf{N}_j) \cdot (\mu_r^{-1} (\nabla \times \mathbf{N}_i)) dV,$$

must be symmetric. This, again, requires $\mu_r^{-1} = (\mu_r^{-1})^T$, which holds for reciprocal media [9], [10], [12].

Unlike $\mathbf{M}$, $\mathbf{K}$ must be positive semi-definite. This implies $\text{Re}\{\mathbf{g}^H \mathbf{K} \mathbf{g}\} \geq 0$ for all nonzero $\mathbf{g}$ [33], which can be tested as:

```
1 eigs_K, _ = eigs(real(K); nev=1, which=:SR)
2 real(eigs_K[1]) > -1e-12
```

Here -10^{-12} is used as the tolerance to allow any numerical zero, even those that ended up negative due to numerical imprecisions. This test passes for all meshes, with μ_r^{-1} being positive definite [9], [12].

To see why $\mathbf{K}$ must be positive semi-definite rather than definite, let us expand the quadratic form:

$$\mathbf{g}^H \mathbf{K} \mathbf{g} = \int_{\Omega} \overline{(\nabla \times \mathbf{E}_h)} \cdot (\mu_r^{-1} (\nabla \times \mathbf{E}_h)) dV.$$

Since it is possible that $\nabla \times \mathbf{E}_h = 0$ for a nonzero field, this implies $\text{Re}\{\mathbf{g}^H \mathbf{K} \mathbf{g}\} \geq 0$.

For the expression above to be zero it is necessary that $\nabla \times \mathbf{E}_h = 0$. Therefore, the number of zero eigenvalues of $\text{Re}\{\mathbf{K}\}$ must be equal to $\dim(\ker(\nabla \times \mathcal{N}_p))$. Consider the curl operator restricted to the Nédélec space on one tetrahedron, $\nabla \times|_{\mathcal{N}_p} : \mathcal{N}_p \rightarrow \nabla \times \mathcal{N}_p$. It is known that $\nabla \times (\nabla q) = 0$ for any polynomial q [9].

Any scalar polynomial $q \in \mathbb{P}_p$ has gradient $\nabla q \in (\mathbb{P}_{p-1})^3$. Recall from the definition of $\mathcal{N}_p$ that $(\mathbb{P}_{p-1})^3 \subset \mathcal{N}_p$, such that $\nabla \mathbb{P}_p \subseteq \ker(\nabla \times|_{\mathcal{N}_p})$. By the exact de Rham complex in equation (2.10), a curl-free polynomial in $\mathcal{N}_p$ must be the gradient of a polynomial in $\mathbb{P}_p$ [9], implying:

$$\ker(\nabla \times|_{\mathcal{N}_p}) = \nabla \mathbb{P}_p.$$

By the rank-nullity theorem [30]:

$$\dim(\nabla \mathbb{P}_p) = \dim(\mathbb{P}_p) - \dim(\ker(\nabla|_{\mathbb{P}_p})).$$

Clearly, the only polynomials with zero gradient are the constant polynomials, giving $\dim(\ker(\nabla|_{\mathbb{P}_p})) = 1$.

To evaluate $\dim(\mathbb{P}_p)$, note that every monomial $x_1^{\alpha_1} x_2^{\alpha_2} x_3^{\alpha_3} \in \mathbb{P}_p(\mathbb{R}^3)$ is uniquely identified by a multi-index $\alpha = (\alpha_1, \alpha_2, \alpha_3) \in (\mathbb{N}_0)^3$. Then, $\mathbb{P}_p$ is isomorphic to the set of exponents:

$$\mathbb{P}_p \cong \left\{ \alpha \in (\mathbb{N}_0)^3 \mid \sum_{i=1}^3 \alpha_i \leq p \right\}.$$

The cardinality of this set can be found by introducing a slack variable $\alpha_4 = p - \sum_{i=1}^3 \alpha_i \geq 0$, transforming the inequality into:

$$\sum_{i=1}^4 \alpha_i = p.$$

By the stars and bars theorem [34], the number of non-negative integer solutions to placing p items into 4 bins is given by the combination:

$$\dim(\mathbb{P}_p) = \binom{p+4-1}{4-1} = \binom{p+3}{3} = \frac{(p+1)(p+2)(p+3)}{6}.$$

Substituting this result back into the rank-nullity expression yields the dimension of the local kernel space:

$$\dim(\ker(\nabla \times|_{\mathcal{N}_p})) = \binom{p+3}{3} - 1.$$

For $p = 2$ on a tetrahedron, $\dim(\ker(\nabla \times|_{\mathcal{N}_2})) = 9$, which coincides with the number of vertices plus the number of edges minus one.

While the local kernel dimension gives the nullity on a single element K , the global number of zero eigenvalues of $\mathbf{K}$ depends on how these local gradients join together globally across the entire mesh Ω .

A global discrete field $\mathbf{E}_h$ is in the kernel of $\mathbf{K}$ if, and only if, $\nabla \times \mathbf{E}_h = 0$. By the exact de Rham sequence, this implies that $\mathbf{E}_h$ must be the gradient of a scalar field $\phi_h \in H_h^1(\Omega)$ [9]:

$$\mathbf{E}_h = \nabla \phi_h,$$

with $H_h^1(\Omega) = \{\phi \in H^1(\Omega) \mid \phi|_K \in \mathbb{P}_p, \forall K \in \mathcal{T}_h\}$. For $p = 2$, $\dim(H_h^1(\Omega))$ is equal to the total number of vertices and edges in the mesh $\mathcal{T}_h$ minus 1 [9]. In Julia, this is tested using:

```

1 NVert = length(nodes)
2 NZero = NVert + NEdge - 1
3
4 eigs_K, _ = eigs(real(K); nev=(NZero + 1), sigma=1e-8)
5 count(abs.(real.(eigs_K)) .< 1e-12) # Must be equal to NZero

```

Here N_{Edge} was returned by `generate_dof_mapping`, and an extra eigenvalue is computed to make sure the N_{Zero} -th is the last null eigenvalue. For mesh 1 N_{Zero} is 364, and for mesh 2 it is 2456, correctly coinciding with the computed number of zero eigenvalues. For meshes 3 and 4, computing the eigenvalues of such large systems was unfeasible given the computational resources.

4.2. Method of Manufactured Solutions

The method of manufactured solutions (MMS) is a method used to verify that a FEM implementation is functioning as it should [29]. It consists of choosing a manufactured solution $\mathbf{V}_{\text{MMS}}$ (for which an analytical expression is available), and plugging it into the weak problem to solve backwards. Consider the differential equation:

$$\nabla^2 \mathbf{E} = \mathbf{J}. \quad (4.1)$$

Setting $\mathbf{J} = \nabla^2 \mathbf{V}_{\text{MMS}}$, and solving the weak form of equation (4.1) using that source vector, yields an approximation of $\mathbf{V}_{\text{MMS}}$. Assessing the error between the approximation and the manufactured solution gives an estimate of the accuracy of the code and helps catch implementation errors. Three tests are considered, as described in [29]:

Test 1

The first equation to solve is:

$$\mathbf{E} = \mathbf{V}_{\text{MMS}}.$$

Note that this is just a projection problem rather than a differential equation. The variational form of this problem reads: find $\mathbf{E} \in \mathbf{V}(\Omega)$ such that

$$\int_{\Omega} \bar{\mathbf{v}} \cdot \mathbf{E} \, dV = \int_{\Omega} \bar{\mathbf{v}} \cdot \mathbf{V}_{\text{MMS}} \, dV, \quad \forall \mathbf{v} \in \mathbf{V}(\Omega).$$

The left side of the equation is just the mass matrix as described in section 2.1.4. The right-hand side can be discretized as:

$$\int_K \mathbf{N}_j^\top \mathbf{V}_{\text{MMS}} \, dV = \int_{\hat{K}} \hat{\mathbf{N}}_j^\top [\mathbf{J}_K^{-\top} \det(\mathbf{J}_K)] (\mathbf{V}_{\text{MMS}} \circ \Phi_K) \, d\hat{V}.$$

This is computed as:

```
1  M((x̂, ŷ, ž) -> transpose(N̂ᵀ(SVector(x̂, ŷ, ž))) * transpose(invJ) *
    ↪ V_mms(nodes[element[1]] + transpose(J) * SVector(x̂, ŷ, ž)) * abs(detJ), 1e-12,
    ↪ ComplexF64)
```

Given an approximation $\mathbf{E}_h \in \mathbf{V}_h(\Omega)$, the chosen error measure is:

$$e_f(\mathbf{V}_{\text{MMS}}, \mathbf{E}_h) = \frac{\sqrt{\int_{\Omega} \|\mathbf{V}_{\text{MMS}} - \mathbf{E}_h\|_2^2 \, dV}}{\sqrt{\int_{\Omega} \|\mathbf{V}_{\text{MMS}}\|_2^2 \, dV}}.$$

That is, the relative error in the fields themselves. The error metric from [29] can be problematic if at any point $\mathbf{V}_{\text{MMS}}$ is zero, because the whole metric gets invalidated by a division-by-zero floating-point exception. Pushing the integral inside the fraction removes this instability. The following Julia function computes this error:

```

1 function mms_e_f_standard(V_mms, B, nodes, elements, c)
2     num = 0.0
3     den = 0.0
4
5     for (e, element) in enumerate(elements)
6         K = Tetrahedron(SMatrix{3,4,Float64}([
7             nodes[element[1]];
8             nodes[element[2]];
9             nodes[element[3]];
10            nodes[element[4]]
11        ]))
12        J = jacobian(K)
13        detJ = det(J)
14        invJ = inv(J)
15
16        N̂ = Nedelec(K, SVector(element), Val(2))
17
18        signs = compute_signs(K)
19
20        num += ∫((x̂, ŷ, ẑ) -> begin
21            V_h = zeros(Float64, 3)
22            for i in 1:20
23                V_h += c[B[e, i]] * N̂[i](SVector(x̂, ŷ, ẑ)) * signs[i]
24            end
25            V_h = invJ * V_h
26
27            V_mms_val = V_mms(nodes[element[1]] + transpose(J) * SVector(x̂, ŷ,
28                ↪ ẑ))
29
30            return norm(V_mms_val - V_h)^2
31        end, 1e-12, Float64) * abs(detJ)
32
33        den += ∫((x̂, ŷ, ẑ) -> begin
34            V_mms_val = V_mms(nodes[element[1]] + transpose(J) * SVector(x̂, ŷ,
35                ↪ ẑ))
36
37            return norm(V_mms_val)^2
38        end, 1e-12, Float64) * abs(detJ)
39    end
40
41    if den == 0.0
42        return sqrt(num)
43    else
44        return sqrt(num) / sqrt(den)
45    end
46 end

```

Test 2

The second test aims to verify the correct implementation of the curls of the basis functions by solving the equation:

$$\nabla \times \mathbf{E} = \nabla \times \mathbf{V}_{\text{MMS}}.$$

The weak form of this problem is: find $\mathbf{E} \in \mathbf{V}(\Omega)$ such that

$$\int_{\Omega} (\nabla \times \bar{\mathbf{v}}) \cdot (\nabla \times \mathbf{E}) dV = \int_{\Omega} (\nabla \times \bar{\mathbf{v}}) \cdot (\nabla \times \mathbf{V}_{\text{MMS}}) dV, \quad \forall \mathbf{v} \in \mathbf{V}(\Omega).$$

The expression on the left-hand side is the stiffness matrix from section 2.1.4, and the right-hand side can be discretized as:

$$\int_K (\nabla \times \mathbf{N}_j)^\top (\nabla \times \mathbf{V}_{\text{MMS}}) dV = \int_{\hat{K}} (\hat{\nabla} \times \hat{\mathbf{N}}_j)^\top \mathbf{J}_K (\nabla \times \mathbf{V}_{\text{MMS}} \circ \boldsymbol{\Phi}_K) d\hat{V}.$$

This is computed as:

```
1  ⚡((x̂, ŷ, ẑ) -> transpose(Ň i(SVector(x̂, ŷ, ẑ))) * J * ∇xV_mms(nodes[element[1]] +
    ↪ transpose(J) * SVector(x̂, ŷ, ẑ)), 1e-12, ComplexF64) * signs[i]
```

The error measure here is:

$$e_c(\mathbf{V}_{\text{MMS}}, \mathbf{E}_h) = \frac{\sqrt{\int_{\Omega} \|\nabla \times \mathbf{V}_{\text{MMS}} - \nabla \times \mathbf{E}_h\|_2^2 dV}}{\sqrt{\int_{\Omega} \|\nabla \times \mathbf{V}_{\text{MMS}}\|_2^2 dV}}.$$

That is, the relative error in the curls of the fields. As before, the measure used is adapted from [29] to avoid the zero-point problem. The following Julia function computes this error:

```
1  function mms_e_c_standard(∇xV_mms, B, nodes, elements, c)
2      num = 0.0
3      den = 0.0
4
5      for (e, element) in enumerate(elements)
6          K = Tetrahedron(SMatrix{3,4,Float64}([
7              nodes[element[1]];
8              nodes[element[2]];
9              nodes[element[3]];
10             nodes[element[4]]
11         ]))
12         J = jacobian(K)
13         detJ = det(J)
14
15         Ň = Nedelec(K, SVector(element), Val(2))
16         ∇xŇ = curl(Ň)
17
18         signs = compute_signs(K)
```

```

19
20     num +=  $\mathbb{I}((\hat{x}, \hat{y}, \hat{z}) \rightarrow \text{begin}$ 
21          $\nabla xV\_h = \text{zeros}(\text{Float64}, 3)$ 
22         for i in 1:20
23              $\nabla xV\_h += c[B[e, i]] * \nabla x\hat{N}[i](\text{SVector}(\hat{x}, \hat{y}, \hat{z})) * \text{signs}[i]$ 
24         end
25          $\nabla xV\_h = \text{transpose}(J) * \nabla xV\_h / \text{det}J$ 
26
27          $\nabla xV\_mms\_val = \nabla xV\_mms(\text{nodes}[\text{element}[1]] + \text{transpose}(J) * \text{SVector}(\hat{x},$ 
28              $\hookrightarrow \hat{y}, \hat{z}))$ 
29
30         return norm( $\nabla xV\_mms\_val - \nabla xV\_h$ )^2
31     end, 1e-12, Float64) * abs(detJ)
32
33     den +=  $\mathbb{I}((\hat{x}, \hat{y}, \hat{z}) \rightarrow \text{begin}$ 
34          $\nabla xV\_mms\_val = \nabla xV\_mms(\text{nodes}[\text{element}[1]] + \text{transpose}(J) * \text{SVector}(\hat{x},$ 
35              $\hookrightarrow \hat{y}, \hat{z}))$ 
36
37         return norm( $\nabla xV\_mms\_val$ )^2
38     end, 1e-12, Float64) * abs(detJ)
39
40     if den == 0.0
41         return sqrt(num)
42     else
43         return sqrt(num) / sqrt(den)
44     end
45 end

```

Test 3

The final test combines the previous two to solve a full double-curl equation:

$$\nabla \times (\nabla \times \mathbf{E}) - k_0^2 \mathbf{E} = \nabla \times (\nabla \times \mathbf{V}_{\text{MMS}}) - k_0^2 \mathbf{V}_{\text{MMS}}.$$

This yields the weak problem: find $\mathbf{E} \in \mathbf{V}(\Omega)$ such that

$$\int_{\Omega} (\nabla \times \bar{\mathbf{v}}) \cdot (\nabla \times \mathbf{E}) dV - k_0^2 \int_{\Omega} \bar{\mathbf{v}} \cdot \mathbf{E} dV = \int_{\Omega} (\nabla \times \bar{\mathbf{v}}) \cdot (\nabla \times \mathbf{V}_{\text{MMS}}) dV - k_0^2 \int_{\Omega} \bar{\mathbf{v}} \cdot \mathbf{V}_{\text{MMS}} dV.$$

The left-hand side is just $\mathbf{K} - k_0^2 \mathbf{M}$, and the right-hand side discretized is given by:

$$\begin{aligned} & \int_K (\nabla \times \mathbf{N}_j)^\top (\nabla \times \mathbf{V}_{\text{MMS}}) dV - k_0^2 \int_K \mathbf{N}_j^\top \mathbf{V}_{\text{MMS}} dV \\ &= \int_{\hat{K}} (\hat{\nabla} \times \hat{\mathbf{N}}_j)^\top \mathbf{J}_K (\nabla \times \mathbf{V}_{\text{MMS}} \circ \Phi_K) d\hat{V} - k_0^2 \int_{\hat{K}} \hat{\mathbf{N}}_j^\top [\mathbf{J}_K^{-\top} \det(\mathbf{J}_K)] (\mathbf{V}_{\text{MMS}} \circ \Phi_K) d\hat{V}. \end{aligned}$$

Note that this is just the integrals from the first two tests summed together.

For the error metric the sum of the errors defined above is used [29]:

$$e(\mathbf{V}_{\text{MMS}}, \mathbf{E}_h) = e_f(\mathbf{V}_{\text{MMS}}, \mathbf{E}_h) + e_c(\mathbf{V}_{\text{MMS}}, \mathbf{E}_h).$$

4.2.1. Functions in the Function Space (Error Test)

To test the algebraic correctness of the code, let us select our manufactured solution to be a polynomial in the basis space, $\mathbf{V}_{\text{MMS}} \in \mathcal{N}_2$. Since this function can be perfectly represented using the basis functions, the expected error is a numerical zero. The results of running this test on the first three meshes for all three tests, using [29]:

$$\mathbf{V}_{\text{MMS}}(\mathbf{x}) = \begin{bmatrix} y^2 \\ -xy \\ 0 \end{bmatrix},$$

are specified in table 4.1. Furthermore, a fringe plot showcasing the results of test 3 on mesh 3 are shown in 4.2. Mesh 4 was excluded from this test as an iterative solver was required due to memory constraints. This solver would have introduced errors orders of magnitude larger than expected, polluting the results.

Table 4.1. RELATIVE ERRORS OF THE MMS WITH A POLYNOMIAL SOLUTION.

	Test 1	Test 2	Test 3
Mesh 1	1.4891×10^{-15}	1.6140×10^{-13}	2.9665×10^{-13}
Mesh 2	1.5508×10^{-15}	3.1853×10^{-13}	3.9999×10^{-13}
Mesh 3	1.5853×10^{-15}	6.4789×10^{-12}	2.0983×10^{-13}

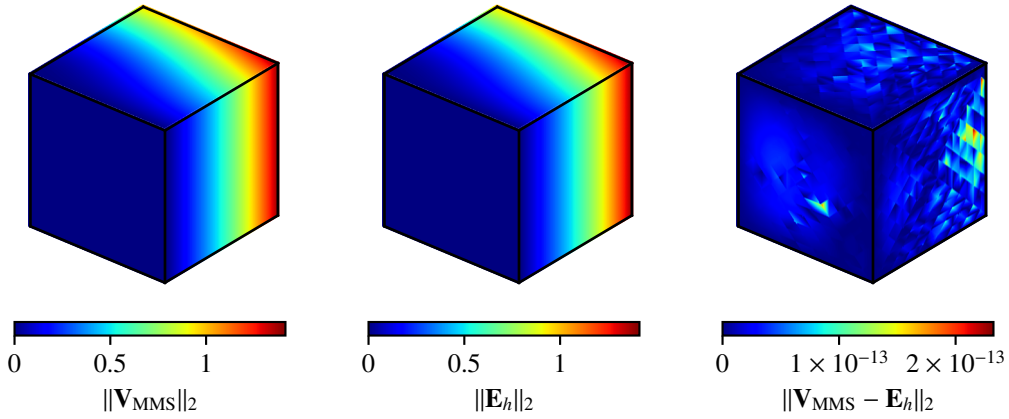


Fig. 4.2. Fringe plots of the result of running the MMS test 3 on mesh 3 with a polynomial solution.

These results showcase the algebraic correctness of the implementation. The rise in error from test 1 to tests 2 and 3 is due to the stiffness matrix $\mathbf{K}$; its inherent ill-conditioning amplifies floating-point errors, restricting the solver to a lower precision.

4.2.2. Functions not in the Function Space (Convergence Test)

While testing functions within the finite element space verifies the algebraic correctness of the implementation, it does not verify the convergence properties of the discretized error. The behavior of this error as the mesh is refined (i.e., as $h \rightarrow 0$) is dictated by two fundamental theorems in FEM theory [5], [9]:

1. **Céa's Lemma:** This lemma states that the error of the numerical solution $\mathbf{E}_h$ in the $\mathbf{H}(\text{curl}, \Omega)$ norm is bounded by a constant multiple of the best possible approximation error within the chosen finite element space $\mathbf{V}_h(\Omega)$:

$$\|\mathbf{V}_{\text{MMS}} - \mathbf{E}_h\|_{\mathbf{H}(\text{curl}, \Omega)} \leq C \inf_{\mathbf{v}_h \in \mathbf{V}_h(\Omega)} \|\mathbf{V}_{\text{MMS}} - \mathbf{v}_h\|_{\mathbf{H}(\text{curl}, \Omega)},$$

where C is a constant independent of h .

2. **Bramble-Hilbert Lemma:** This lemma bounds the right-hand side of Céa's Lemma. By defining an interpolation operator $\Pi_h : \mathbf{H}(\text{curl}, \Omega) \rightarrow \mathbf{V}_h(\Omega)$, the lemma limits the interpolation error based on the polynomial degree of the basis functions and the smoothness of the exact solution.

Combining both lemmas yields an a priori error estimate. For Nédélec elements of the first kind of order p , the theoretical error bound in the $\mathbf{H}(\text{curl}, \Omega)$ norm is given by [9]:

$$\|\mathbf{V}_{\text{MMS}} - \mathbf{E}_h\|_{\mathbf{H}(\text{curl}, \Omega)} \leq Ch^p (\|\mathbf{V}_{\text{MMS}}\|_{\mathbf{H}^p(\Omega)} + \|\nabla \times \mathbf{V}_{\text{MMS}}\|_{\mathbf{H}^p(\Omega)}) = O(h^p). \quad (4.2)$$

For this implementation using second-order Nédélec tetrahedra of the first kind, the error is expected to converge at a rate of $O(h^2)$. Taking the logarithm of both sides of equation (4.2) gives:

$$\log(\|\mathbf{V}_{\text{MMS}} - \mathbf{E}_h\|_{\mathbf{H}(\text{curl}, \Omega)}) \leq p \log(h) + \log(C) + \log(\|\mathbf{V}_{\text{MMS}}\|_{\mathbf{H}^p(\Omega)} + \|\nabla \times \mathbf{V}_{\text{MMS}}\|_{\mathbf{H}^p(\Omega)}),$$

with $p = 2$. Therefore, when plotting the computed errors against the mesh refinement h on a log-log scale, the data points are expected to form a straight line with a slope of approximately 2. To verify this, a complex exponential is chosen as the manufactured solution [29]:

$$\mathbf{V}_{\text{MMS}}(\mathbf{x}) = \begin{bmatrix} (\cos(\theta) \cos(\phi) - \sin(\phi)) e^{-ik_0(\mathbf{k} \cdot \mathbf{x})} \\ (\cos(\theta) \sin(\phi) + \cos(\phi)) e^{-ik_0(\mathbf{k} \cdot \mathbf{x})} \\ -\sin(\theta) e^{-ik_0(\mathbf{k} \cdot \mathbf{x})} \end{bmatrix}, \quad \mathbf{k} = \begin{bmatrix} -\sin(\theta) \cos(\phi) \\ -\sin(\theta) \sin(\phi) \\ -\cos(\theta) \end{bmatrix},$$

with $\theta = 20^\circ$, $\phi = 30^\circ$, and $k_0 = 2\pi$. This function is ideal as it is smooth and not in the basis space, meaning a non-zero error is expected. Then, table 4.2 specifies the results of running this test on all meshes, and figure 4.3 illustrates the results for test 3. From the slopes listed in the last row of the table it is clear that all tests present the desired convergence. Here mesh 4 could be included in the results as the error induced by the iterative solver was still orders of magnitude smaller than the full error.

Table 4.2. RELATIVE ERRORS OF THE MMS WITH AN EXPONENTIAL SOLUTION.

	Test 1	Test 2	Test 3
Mesh 1	0.070132	0.094966	0.186892
Mesh 2	0.017448	0.024917	0.047917
Mesh 3	0.004287	0.006315	0.012162
Mesh 4	0.001059	0.001585	0.003062
Slope	2.02	1.97	1.98

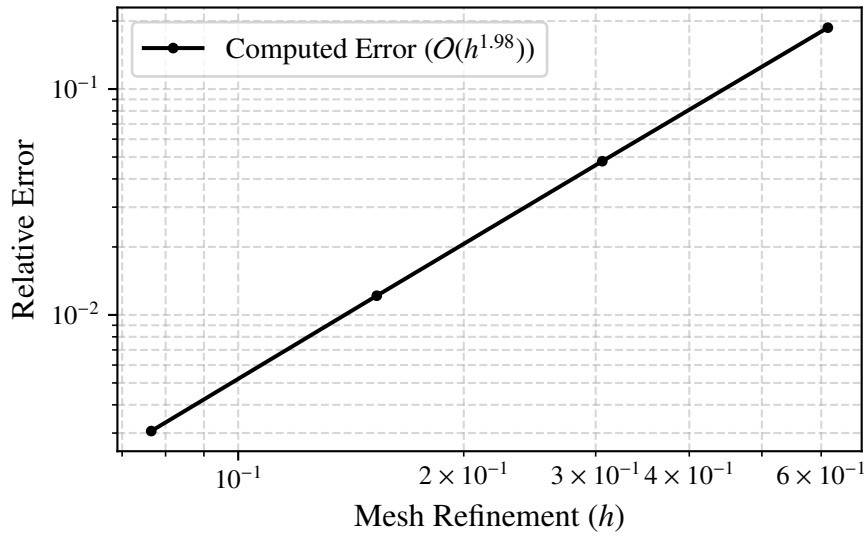


Fig. 4.3. Convergence of the relative error in the MMS test 3.

Finally, to show the improvement from mesh 1 to mesh 4, the fringe plots of all meshes are shown in figures 4.4, 4.5, 4.6 and 4.7.

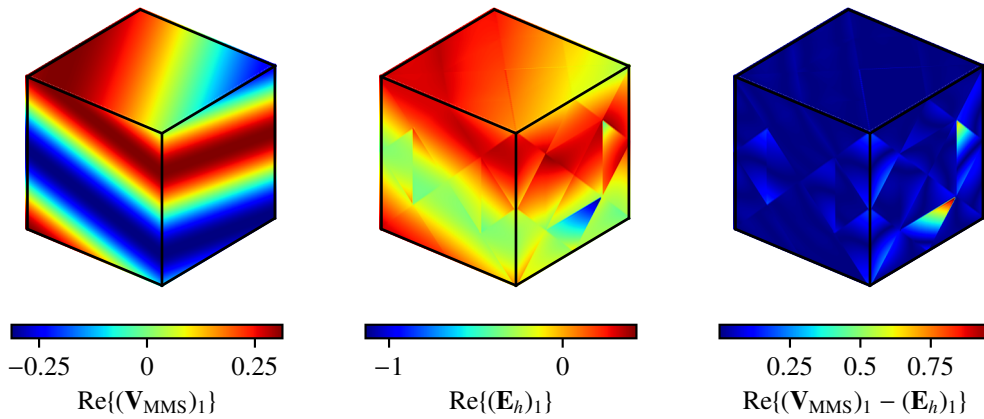


Fig. 4.4. Fringe plots of the result of running the MMS test 3 on mesh 1 with a complex exponential solution.

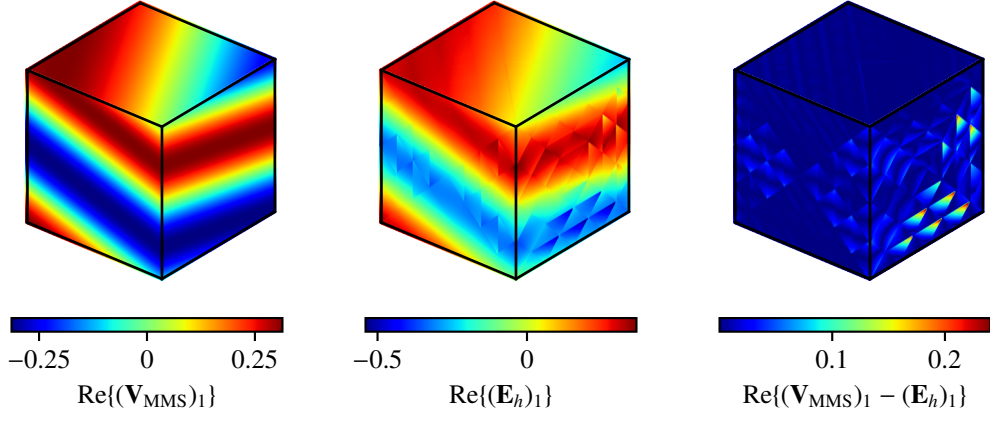


Fig. 4.5. Fringe plots of the result of running the MMS test 3 on mesh 2 with a complex exponential solution.

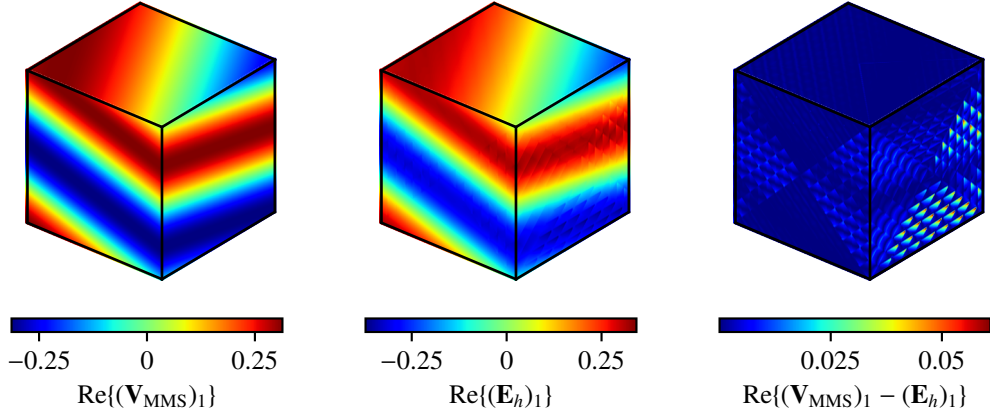


Fig. 4.6. Fringe plots of the result of running the MMS test 3 on mesh 3 with a complex exponential solution.

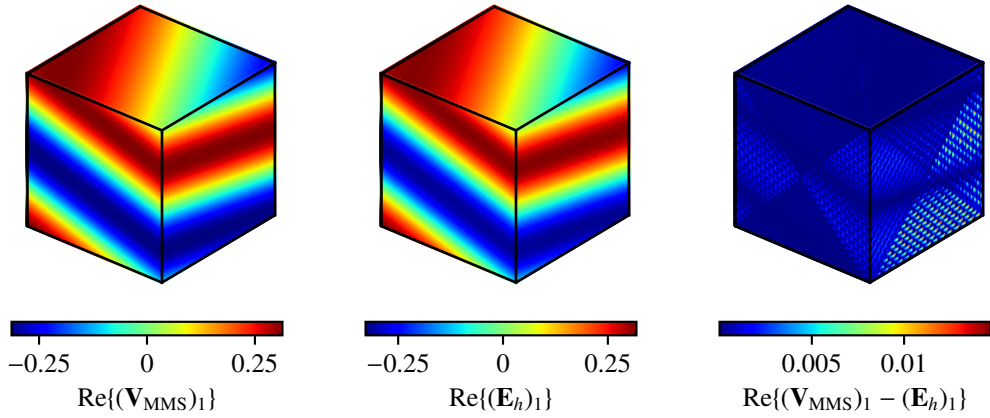


Fig. 4.7. Fringe plots of the result of running the MMS test 3 on mesh 4 with a complex exponential solution.

4.3. HOFEM Test

Following the line of investigation leading to this thesis, interaction between Julia and HOFEM was tested. Two separate, but related tests were performed. The first test consisted of running HOFEM step-by-step using the Julia interface, and the second test involved skipping the assembly steps, passing the Julia-generated system to HOFEM, and solving it instead of the one HOFEM would have generated. But before explaining the tests and their results, some context regarding the interface is needed.

The interface [4] consists of two different systems. The first system is a parser that takes in Fortran source files and generates bindings that can be called from Julia by using the `@ccall` macro (the Julia functions calling the Fortran bindings are also generated). The second system is the generated interface itself. For the tests only the latter is relevant.

As of the current version, the generated interface is organized so that every simulation step can be run using `hofem_run_simulation`. This function receives a project name for finding the relevant project files, a program mode stating which execution mode to run in (frequency sweep, adaptive mesh analysis, etc.), a step flag indicating which steps to run, and indices stating the start and end of the frequencies (only frequency sweep is fully supported). Then, running HOFEM step by step is as simple as calling `hofem_run_simulation` with individual steps:

```
1 ENV["HOFEM_LIB_PATH"] = "path/to/libHOFEM.so"
2
3 include("src/Interfaces/Interface.jl")
4
5 project_name      = "pyramid"
6 program_mode      = "FREQ-SWEEP"
7 postprocess_mode  = ""
8
9 # Include all frequencies
10 freq_start_idx    = Cint(-1)
11 freq_end_idx      = Cint(-1)
12
13 Interface.hofem_run_simulation(project_name, program_mode,
14     ↪ Interface.LIB_STEP_INITIALIZATION(), postprocess_mode, freq_start_idx,
15     ↪ freq_end_idx)
16 Interface.hofem_run_simulation(project_name, program_mode,
17     ↪ Interface.LIB_STEP_INPUT_FILE_READ(), postprocess_mode, freq_start_idx,
18     ↪ freq_end_idx)
19 Interface.hofem_run_simulation(project_name, program_mode, Interface.LIB_STEP_MESH(),
20     ↪ postprocess_mode, freq_start_idx, freq_end_idx)
21 Interface.hofem_run_simulation(project_name, program_mode,
22     ↪ Interface.LIB_STEP_ASSEMBLY_LHS() + Interface.LIB_FLAG_ASSEMBLY_DIRICHLET(),
23     ↪ postprocess_mode, freq_start_idx, freq_end_idx)
24 Interface.hofem_run_simulation(project_name, program_mode,
25     ↪ Interface.LIB_STEP_ASSEMBLY_RHS() + Interface.LIB_FLAG_ASSEMBLY_DIRICHLET(),
26     ↪ postprocess_mode, freq_start_idx, freq_end_idx)
27 Interface.hofem_run_simulation(project_name, program_mode,
28     ↪ Interface.LIB_STEP_FACTORIZE(), postprocess_mode, freq_start_idx, freq_end_idx)
```

```

19 Interface.hofem_run_simulation(project_name, program_mode,
    ↳ Interface.LIB_STEP_SOLVE(), postprocess_mode, freq_start_idx, freq_end_idx)
20 Interface.hofem_run_simulation(project_name, program_mode,
    ↳ Interface.LIB_STEP_FINALIZATION(), postprocess_mode, freq_start_idx,
    ↳ freq_end_idx)

```

For two different projects, “pyramid” and “one tet”, the above code ran as expected yielding the same results as running HOFEM directly.

The second test involved skipping some of the steps and inserting the Julia-assembled matrix to be solved by HOFEM. For simplicity’s sake and to avoid any integration issues, the Julia matrix was saved in COO format in three CSV files, rows.csv, cols.csv and data.csv. The right hand side was saved in b.csv. The following code was written for the test:

```

1  matrix_path = "path/to/files"
2  IRN = vec(readdlm(joinpath(matrix_path, "rows.csv"), ', ', Int32))
3  JCN = vec(readdlm(joinpath(matrix_path, "cols.csv"), ', ', Int32))
4  V = vec(readdlm(joinpath(matrix_path, "data.csv"), ', ', Float64))
5  A = sparse(IRN, JCN, V)
6  IRN, JCN, V = findnz(triu(A))
7  IRN = Vector{Cint}(IRN)
8  JCN = Vector{Cint}(JCN)
9  nz = length(IRN)
10 b = vec(readdlm(joinpath(matrix_path, "b.csv"), ', ', ComplexF64))
11 N = 10712 # Loaded system is 10712 by 10712
12
13 solution = A \ b
14
15 steps = Interfaces.LIB_STEP_INITIALIZATION()
16
17 Interfaces.hofem_run_simulation("one-tet", "FREQ-SWEEP", steps, "", Cint(-1),
    ↳ Cint(-1))
18
19 mumpsObj = Interfaces.get_MUMPS_OBJ()
20 Interfaces.set_N!(mumpsObj, Cint(N))
21 Interfaces.set_NNZ!(mumpsObj, Cint(nz))
22 Interfaces.set_NZ_loc!(mumpsObj, Cint(nz))
23 Interfaces.set_A_loc!(mumpsObj, pointer(V), Cint(nz))
24 Interfaces.set_IRN_loc!(mumpsObj, pointer(IRN), Cint(nz))
25 Interfaces.set_JCN_loc!(mumpsObj, pointer(JCN), Cint(nz))
26 Interfaces.set_RHS!(mumpsObj, pointer(b), Cint(N))
27
28 Interfaces.hofem_run_simulation("one-tet", "FREQ-SWEEP",
    ↳ Interfaces.LIB_STEP_FACTORIZE() | Interfaces.LIB_STEP_SOLVE(), "", Cint(-1),
    ↳ Cint(-1))
29
30 solution_HOFEM_ptr = Interfaces.get_RHS(mumpsObj)
31 solution_HOFEM = unsafe_wrap(Array, solution_HOFEM_ptr, N, own=false)
32

```

```
33 @info "Relative error:", rel_error(solution_HOFEM, solution)
34
35 Interfaces.hofem_run_simulation("one_tet", "FREQ-SWEEP",
    ↪ Interfaces.LIB_STEP_FINALIZATION(), "", Cint(-1), Cint(-1))
```

Unfortunately, due to last-minute issues and time constraints, the above code could not be tested. However, earlier successful runs of similar scripts suggest it would have also been successful.

5. REGULATORY FRAMEWORK

5.1. Applicable Legislation

Software development involves prolonged use of display screen equipment, which presents specific occupational health risks. In Spain, the Real Decreto 488/1997 de 14 de abril [35] establishes the minimum health and safety requirements for work with display screen equipment. The development of this thesis was carried out minimizing the possible occupational risks.

On an ethical level, professional responsibility is especially important in scientific computing, as incorrect software can lead to faulty engineering designs or unreliable conclusions. For this reason, this thesis follows the ACM/IEEE software engineering code of ethics [36]. As an example, the tests performed in the verification section (4) ensure the correctness of the code developed within the thesis, upholding these ethical standards.

Finally, regarding privacy and security, data protection legislation in Spain is governed by the European GDPR [37] and the Spanish LOPDGDD [38]. The software developed in this thesis only processes physical and mathematical data, which is inherently impersonal, thus automatically complying with both the GDPR and the LOPDGDD.

5.2. Technical Standards

The development of the numerical algorithms within this thesis adheres to several technical standards to ensure reliability, precision, and portability across different systems.

First, the code relies on precise numerical integration and matrix operations. Both Julia and Fortran perform numerical computations following the IEEE 754 standard for floating-point arithmetic [39]. Using this standard guarantees consistency in how numerical values, precision limits, and rounding rules are handled at the hardware level, which is crucial for the rigorous evaluation of basis functions and DoF functionals.

Second, the parallelization of the solver is implemented following the MPI standard [20]. MPI is the most popular standard for distributed-memory parallelization in high-performance computing (HPC). By utilizing an implementation of this standard, the code benefits from its maturity and widespread adoption for its parallelization.

Finally, the programming languages used are developed according to international standards. Specifically, the stability, predictable behavior and performance of HOFEM are guaranteed by the ISO specifications for the Fortran programming language [40].

5.3. Intellectual Property

According to the Real Decreto Legislativo 1/1996 de 12 de abril [41], all software is protected under copyright law the moment it is created. Therefore, the use of third-party software in this thesis is limited to the permissions granted under their respective licenses. Across all packages used, three distinct licenses were identified:

- **MIT [42]:** Permits the free use, copy, modification and distribution of software, but provides no guarantees regarding its reliability or effectiveness.
- **GPL [43]:** Similar to the MIT license, but requiring that any code using GPL-licensed software must also be distributed under this license.
- **Proprietary licenses:** Restrict the use, modification, and distribution of the software to authorized parties.

While the written thesis itself is freely accessible, the software developed is primarily intended for internal use within the GREMA. However, the code developed up to the point of the publication of this thesis has been made available in [44]. Both this release and the software's internal usage fully comply with the limitations of both the MIT and GPL licenses.

Finally, the only proprietary software used in this workflow is HOFEM. Because this thesis was developed directly within the GREMA, explicit permission to access, execute, and interface with the HOFEM codebase was granted by its creators for the development of the thesis.

6. SOCIOECONOMIC ENVIRONMENT

6.1. Cost of Development

The overall costs of developing this thesis can be divided into human capital, material, and software licenses.

Human Capital Costs

In this regard, both the work of the student and the tutors must be considered. In the case of the student, we assume the median entry-level salary for a computational scientist in Madrid, which is 56 087 € per year, or about 32.87 € per hour [45]. University guidelines state that 1 ECTS corresponds to 25 student hours [46]. The subject “TFG”, under which this thesis was developed, amounts to 12 ECTS, giving a total of 300 student hours. Overall, this corresponds to 9861 € of student salary.

In the case of this thesis, two tutors were present: one of them was present at 25 1 hour tutor sessions, and the other at 10, resulting in a total of approximately 35 tutoring hours. The annual salary of a professor at the UC3M is 53 668.28 €, approximately 25.80 € hourly [47]. This sums up to 903 € in tutor salaries.

Material Costs

Little material was needed to carry out this thesis, and none was bought specifically for it. The student’s laptop was a Lenovo IdeaPad-3-14ITL6, featuring an Intel Core i5-1135G7 CPU and 12GB (4GB soldered plus 8GB SO-DIMM DDR4) of RAM, which costed 450 € in the summer of 2022. Amortized over 8 university semesters this results in a cost of 56.25 € per semester.

In the winter of 2024, the 8GB SO-DIMM DDR4 RAM stick was replaced with a 16GB one, costing 50 €. Over 3 semesters, this yields an overall cost of 16.67 € per semester. Since the thesis was developed in one semester, the total material costs amount to 72.92 €.

Software Licenses Cost

As mentioned in section 5.3, Julia and all of the packages used within the implementation are open-source and completely free to use. The only proprietary software used was HOFEM, but since the thesis was developed within the GREMA, the creators provided free access to its codebase. This resulted in no costs related to software.

Total Costs

All costs are summarized and summed in table 6.1.

Table 6.1. BROKEN DOWN COSTS OF DEVELOPING THE THESIS.

Category	Cost (€)
Student (300 hours)	9861
Tutors (35 hours)	903
Human Capital	10 764
Laptop (1 semester)	56.25
RAM (1 semester)	16.67
Material	72.92
Software	0
Total	10 836.92

6.2. Socioeconomic Impact

The main economic impact stemming from this thesis is the mitigation of the two-language problem. Reducing development hours leads to smaller R&D costs, opening space for other projects or less expenditure overall. Furthermore, being able to rely on an extensive ecosystem of scientific tools, such as Julia's, also eases the requirement of implementing fundamental algorithms common to all areas in the field.

Socially, mitigating the two-language problem opens up the field to new researchers and students, allowing more people to contribute in a more timely and effective manner.

This thesis can also be environmentally beneficial. Easily parallelizing software simulations greatly reduces the computational time needed to run them, lowering the energy costs of keeping the computers at full power. More indirectly, improving electromagnetic simulations aids in the creation of more environmentally-friendly devices.

In the same way, several fields ranging from aerospace and defense, telecommunications, and medical technology can take advantage of any advancements in the field to improve their products for the betterment of society.

7. CONCLUSIONS

7.1. Assessment of Objectives

Overall, the aim of this thesis has been accomplished, namely, a Julia-based FEM framework for electromagnetics was developed and successfully verified. The distinct achievements of this framework are discussed below.

First, the framework is able to read and process meshes. This was accomplished by using Gmsh [18] to read the .msh file, later being transformed to a format native to the framework, and generating the DoF mapping from it.

Secondly, the basis functions were computed in a rigorous manner making use of adaptive and non-adaptive numerical integration. Computing them in this way allowed the DoF functionals to be used in alternative parts of the code, such as the enforcement of the Dirichlet boundary conditions.

The third objective was also accomplished by using the previously generated DoF mapping, distinguishing between boundary and interior DoFs. The matrix was assembled using a popular storage format for sparse matrices, to make use of efficient pre-implemented libraries.

Furthermore, MPI [20] was used to parallelize the entire matrix assembly and make the code runnable on multi-processor systems. This parallelization enabled running the code faster and using fewer computational resources.

Finally, the correctness of the code was tested to ensure the reliability of the computations. On one hand, the mathematical properties of the matrix were verified, proving that the system is adequately generated for a given problem. On the other hand, the behavior of the errors was tested, observing that they were correctly bounded and converged according to the theory.

Part of the main motivation the thesis (interacting with HOFEM) was accomplished. The step-by-step execution of HOFEM from Julia was successfully tested, but the solving of a Julia system in HOFEM was not due to time constraints.

The code developed during the thesis can be openly found in [44], contributing to open science and the potential development of the scientific computing field.

7.2. Limitations

Naturally, the scope of this thesis made it unfeasible to cover all possible bases, inevitably leading to limitations in the result.

Perhaps the biggest limitation is considering only Dirichlet boundary conditions. This is because only a partial problem is actually being solved. Therefore, implementing the enforcement of Neumann and Cauchy boundary conditions would make the framework able to solve full electromagnetic problems. In this regard, only small amounts of new code would have to be added as the Neumann and Cauchy boundary conditions are applied in a similar, yet slightly more contrived, manner as the Dirichlet boundary conditions.

Another shortcoming is only allowing tetrahedral meshes. Whilst the tetrahedron is the most popular finite element shape [10], more complex elements, such as prisms or hexahedra, can provide superior accuracy and convergence properties [10].

On a similar note, the framework currently only supports second-order Nédélec elements, limiting error convergence to $O(h^2)$. Adding higher polynomial orders would significantly improve the convergence rate.

Regarding mesh formats, the implementation only allows Gmsh .msh meshes. Although Gmsh is a very popular mesh generation tool, many other exist and are equally widespread. Supporting alternative mesh formats would greatly expand the versatility of the framework.

Only one of the planned tests involving HOFEM was actually performed. Finishing the second test and developing more would help with the goal of interacting more effectively with HOFEM.

7.3. Future Work

Future work following this thesis would involve solving the aforementioned limitations. This would make the framework far more versatile and robust, allowing for even more effective interaction with HOFEM.

Furthermore, current interaction with HOFEM is mostly limited to step-by-step execution. In principle, it is also possible to pass assembled and solved systems between HOFEM and the Julia framework. Ideally, more granular interaction would be achieved, especially being able to share a mesh between both. This would involve either implementing a Gmsh .msh parser in HOFEM, or a GiD mesh parser for the Julia framework.

As a longer term objective, it would be desirable to be able to test HOFEM using the Julia native constructs, and even perform tests such as the MMS. This would conclude the current line of work offering a feature-parity between the frameworks to quickly iterate on (Julia interface) or deploy (HOFEM) new methods.

BIBLIOGRAPHY

- [1] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, *Julia: A fast dynamic language for technical computing*, 2012. arXiv: [1209.5145](https://arxiv.org/abs/1209.5145) [cs.PL].
- [2] The Julia Project, *The julia language ecosystem and packages*, <https://julialang.org>, Accessed: 2026-08, 2026.
- [3] D. García-Doñoro, I. Martínez-Fernández, L. E. García-Castillo, and M. Salazar-Palma, “Hofem: A higher order finite element method electromagnetic simulator,” in *2015 IEEE International Conference on Computational Electromagnetics*, 2015, pp. 10–12. DOI: [10.1109/COMPEM.2015.7052537](https://doi.org/10.1109/COMPEM.2015.7052537).
- [4] D. Mohedano, “Interoperability system between julia and the electromagnetic simulator hofem,” M.S. thesis, Universidad Carlos III de Madrid, 2025.
- [5] S. C. Brenner and L. R. Scott, *The Mathematical Theory of Finite Element Methods*, 3rd. Springer New York, 2008. DOI: [10.1007/978-0-387-75934-0](https://doi.org/10.1007/978-0-387-75934-0).
- [6] G. Strang and G. Fix, *An Analysis of the Finite Element Methods*. Wellesley-Cambridge Press, 2008. DOI: [10.1137/1.9780980232707](https://doi.org/10.1137/1.9780980232707).
- [7] P. G. Ciarlet, *The Finite Element Method for Elliptic Problems*. Society for Industrial and Applied Mathematics, 2002. DOI: [10.1137/1.9780898719208](https://doi.org/10.1137/1.9780898719208).
- [8] A. Bossavit, *Computational Electromagnetism: Variational Formulations, Complementarity, Edge Elements*. Academic Press, 1998. DOI: [10.1016/B978-012118710-1/50010-6](https://doi.org/10.1016/B978-012118710-1/50010-6).
- [9] P. Monk, *Finite Element Methods for Maxwell’s Equations*. Oxford University Press, 2003. DOI: [10.1093/acprof:oso/9780198508885.001.0001](https://doi.org/10.1093/acprof:oso/9780198508885.001.0001).
- [10] J.-M. Jin, *The Finite Element Method in Electromagnetics*, 3rd. IEEE Press / Wiley, 2014. [Online]. Available: <https://www.wiley.com/shop/general-introductory-electrical-electronics-engineering/the-finite-element-method-in-electromagnetics-3rd-edition-p-9781118841983>.
- [11] M. Salazar-Palma, T. K. Sarkar, L.-E. García-Castillo, T. Roy, and A. R. Đorđević, *Iterative and Self-Adaptive Finite-Elements in Electromagnetic Modeling*. Boston, MA: Artech House, 1998.
- [12] L. D. Landau and E. M. Lifshitz, *Electrodynamics of Continuous Media* (Course of Theoretical Physics), 2nd. Pergamon Press, 1984, vol. 8.
- [13] M. Costabel, “A coercive bilinear form for maxwell’s equations,” *Journal of Mathematical Analysis and Applications*, vol. 157, no. 2, pp. 527–541, 1991. DOI: [10.1016/0022-247X\(91\)90104-8](https://doi.org/10.1016/0022-247X(91)90104-8).
- [14] J.-C. Nédélec, “Mixed finite elements in $\mathbb{R}^3$,” *Numerische Mathematik*, vol. 35, pp. 315–341, 1980. DOI: [10.1007/BF01396415](https://doi.org/10.1007/BF01396415).

- [15] R. Hiptmair, “Finite elements in computational electromagnetism,” *Acta Numerica*, vol. 11, pp. 237–339, 2002. DOI: [10.1017/S0962492902000041](https://doi.org/10.1017/S0962492902000041).
- [16] R. Hiptmair, “Canonical construction of finite elements,” *Mathematics of Computation*, vol. 68, no. 228, pp. 1325–1346, 1999. [Online]. Available: <https://www.jstor.org/stable/2584914>.
- [17] A. Buffa, M. Costabel, and D. Sheen, “On traces for $\mathbf{H}(\text{curl}, \Omega)$ in lipschitz domains,” *Journal of Mathematical Analysis and Applications*, vol. 276, no. 2, pp. 845–867, 2002. DOI: [10.1016/S0022-247X\(02\)00455-9](https://doi.org/10.1016/S0022-247X(02)00455-9).
- [18] C. Geuzaine and J.-F. Remacle, “Gmsh: A 3-d finite element mesh generator with built-in pre- and post-processing facilities,” *International Journal for Numerical Methods in Engineering*, vol. 79, no. 11, pp. 1309–1331, 2009. DOI: [10.1002/nme.2579](https://doi.org/10.1002/nme.2579).
- [19] L. F. Demkowicz, P. Monk, L. Vardapetyan, and W. Rachowicz, “De Rham diagram for hp finite element spaces,” *Computers & Mathematics with Applications*, vol. 39, no. 7, pp. 29–38, 2000. DOI: [10.1016/S0898-1221\(00\)00062-6](https://doi.org/10.1016/S0898-1221(00)00062-6).
- [20] Message Passing Interface Forum, “MPI: A message-passing interface standard,” University of Tennessee, Knoxville, TN, USA, Tech. Rep., 1994. DOI: [10.1145/169627.169855](https://doi.org/10.1145/169627.169855).
- [21] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. DOI: [10.1137/141000671](https://doi.org/10.1137/141000671).
- [22] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–86. DOI: [10.1109/CGO.2004.1281665](https://doi.org/10.1109/CGO.2004.1281665).
- [23] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013. DOI: [10.56021/9781421407944](https://doi.org/10.56021/9781421407944).
- [24] The Julia Sparse Contributors, *SparseArrays.jl: Sparse arrays in julia*, <https://juliaarrays.github.io/StaticArrays.jl/stable/>.
- [25] P. J. Davis and P. Rabinowitz, *Methods of Numerical Integration*, 2nd. Academic Press, 1984. DOI: [10.1016/C2013-0-10566-1](https://doi.org/10.1016/C2013-0-10566-1).
- [26] Alex Townsend and contributors, *FastGaussQuadrature.jl: Sparse arrays in julia*, <https://juliaapproximation.github.io/FastGaussQuadrature.jl/stable/>.
- [27] M. G. Duffy, “Quadrature over a pyramid or cube of integrands with a singularity at a vertex,” *SIAM Journal on Numerical Analysis*, vol. 19, no. 6, pp. 1260–1262, 1982. DOI: [10.1137/0719090](https://doi.org/10.1137/0719090).
- [28] B. Szabó and I. Babuška, *Finite Element Analysis*. John Wiley & Sons, 2021. DOI: [10.1002/9781119426479](https://doi.org/10.1002/9781119426479).

- [29] D. García-Donoro, L. E. García-Castillo, and S. W. Ting, “Verification process of finite-element method code for electromagnetics: Using the method of manufactured solutions,” *IEEE Antennas and Propagation Magazine*, vol. 58, no. 2, pp. 28–38, 2016. DOI: [10.1109/MAP.2016.2520308](https://doi.org/10.1109/MAP.2016.2520308).
- [30] S. Axler, *Linear Algebra Done Right* (Undergraduate Texts in Mathematics), Fourth. Springer International Publishing, 2024. DOI: [10.1007/978-3-031-41026-0](https://doi.org/10.1007/978-3-031-41026-0).
- [31] S. Byrne, L. C. Wilcox, and V. Churavy, “MPI.jl: Julia bindings for the Message Passing Interface,” *Proceedings of the JuliaCon Conferences*, vol. 1, no. 1, p. 68, 2021. DOI: [10.21105/jcon.00068](https://doi.org/10.21105/jcon.00068).
- [32] P. R. Amestoy, I. S. Duff, J.-Y. L’Excellent, and J. Koster, “Mumps: A general purpose distributed memory sparse solver,” in *Applied Parallel Computing. New Paradigms for HPC in Industry and Academia*, Springer Berlin Heidelberg, 2001, pp. 121–130. DOI: [10.1007/3-540-70734-4_16](https://doi.org/10.1007/3-540-70734-4_16).
- [33] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, Second. Society for Industrial and Applied Mathematics, 2002. DOI: [10.1137/1.9780898718027](https://doi.org/10.1137/1.9780898718027).
- [34] W. Feller, *An Introduction to Probability Theory and Its Applications, Volume 1*, Third. John Wiley & Sons, 1968.
- [35] Ministerio de Trabajo y Asuntos Sociales, *Real decreto 488/1997, de 14 de abril, sobre disposiciones mínimas de seguridad y salud relativas al trabajo con equipos que incluyen pantallas de visualización*, 1997. [Online]. Available: <https://www.boe.es/buscar/act.php?id=BOE-A-1997-8671>.
- [36] D. Gotterbarn, K. Miller, and S. Rogerson, “Software engineering code of ethics,” *Commun. ACM*, vol. 40, no. 11, pp. 110–118, 1997. DOI: [10.1145/265684.265699](https://doi.org/10.1145/265684.265699). [Online]. Available: <https://www.acm.org/code-of-ethics/software-engineering-code>.
- [37] European Union, *Regulation (eu) 2016/679 ... (general data protection regulation)*, Accessed: 2026-08, 2016. [Online]. Available: <https://gdpr.eu/>.
- [38] Jefatura del Estado, *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, 2018. [Online]. Available: <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>.
- [39] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic*. IEEE, 2019, IEEE Std 754-2019 (Revision of IEEE 754-2008). DOI: [10.1109/IEEESTD.2019.8766229](https://doi.org/10.1109/IEEESTD.2019.8766229).
- [40] ISO/IEC JTC 1/SC 22, “Information technology — Programming languages — Fortran — Part 1: Base language,” International Organization for Standardization, Standard ISO/IEC 1539-1:2023, 2023. [Online]. Available: <https://www.iso.org/standard/82170.html>.

- [41] Ministerio de Cultura, *Real decreto legislativo 1/1996, de 12 de abril, por el que se aprueba el texto refundido de la ley de propiedad intelectual, regularizando, aclarando y armonizando las disposiciones legales vigentes sobre la materia*, 1996. [Online]. Available: <https://www.boe.es/buscar/act.php?id=BOE-A-1996-8930>.
- [42] T. O. S. Initiative, *The mit license*, Open Source Initiative approved license, 1987. [Online]. Available: <https://opensource.org/license/mit>.
- [43] T. O. S. Initiative, *The gnu general public license version 3*, Open Source Initiative approved license, 2007. [Online]. Available: <https://opensource.org/license/gpl-3.0>.
- [44] S. Madrid Cuevas, *Bachelor's thesis software: Parallel programming of fem in julia*, 2026. DOI: [10.5281/zenodo.22638770](https://doi.org/10.5281/zenodo.22638770).
- [45] SalaryExpert, *Computational scientist salary in Madrid, Spain (2026)*, <https://www.salaryexpert.com/salary/job/computational-scientist/spain/madrid>, Accessed: 2026-08, 2026.
- [46] Universidad Carlos III de Madrid, *Evaluación continua*, https://www.uc3m.es/ss/Satellite/Grado/es/TextoMixta/1371214034625/Evaluacion_continua, Accessed: 2026-08.
- [47] Universidad Carlos III de Madrid, *Retribuciones*, <https://www.uc3m.es/conocenos/recursos-humanos/retribuciones>, Accessed: 2026-08.